# CHƯƠNG 7

# THIẾT KẾ HỆ THỐNG VÀ DỮ LIỆU CHO ỨNG DỤNG AI TẠO SINH

*Sau khi học xong chương này, người học có thể:*

- *Mô tả được sự khác biệt căn bản về tổ chức dữ liệu giữa ứng dụng truyền thống và ứng dụng AI tạo sinh, cũng như bảy nguyên tắc thiết kế kiến trúc dữ liệu cho GenAI.*
- *Phân tích được mô hình ba tầng dữ liệu (raw database, vector database, private data enclaves) và giải thích được vai trò của vector database trong việc bổ sung – không thay thế – cơ sở dữ liệu quan hệ truyền thống.*
- *Giải thích được nguyên lý hoạt động của mô hình RAG, so sánh được ba mức độ phát triển của ứng dụng hỏi đáp và phân biệt được Traditional RAG với Agentic RAG.*
- *Thiết kế được pipeline dữ liệu cho ứng dụng AI tạo sinh, bao gồm các giai đoạn từ thu thập, tiền xử lý, chunking, embedding, lưu trữ, truy xuất đến sinh nội dung.*
- *Áp dụng được các kỹ thuật tối ưu hóa hiệu năng (pruning, quantization, knowledge distillation) và các chiến lược vận hành (batch processing, caching, model routing, tiered storage).*

## 7.1 Tổ chức và quản lý dữ liệu trong ứng dụng AI

*Sau khi hoàn thành mục này, sinh viên có thể:*

- *Mô tả được sự khác biệt căn bản về tổ chức dữ liệu giữa ứng dụng truyền thống và ứng dụng AI tạo sinh.*
- *Trình bày được bảy nguyên tắc thiết kế kiến trúc cho ứng dụng AI tạo sinh.*
- *Phân tích được mô hình ba tầng dữ liệu và vai trò của từng tầng trong hệ thống AI.*
- *Giải thích được các chiến lược tổ chức dữ liệu phi cấu trúc, bao gồm chunking và embedding.*
- *Nhận diện được các miền hạ tầng cốt lõi cần thiết cho ứng dụng AI tạo sinh.*

### *7.1.1 Khởi động: Dữ liệu trong ứng dụng AI có gì khác?*

Hãy hình dung bạn đang thiết kế cơ sở dữ liệu cho một ứng dụng quản lý bán hàng. Bạn nghĩ ngay đến các bảng: `khach\_hang`, `don\_hang`, `san\_pham` – mỗi bảng có các cột được định nghĩa rõ ràng, các mối quan hệ khóa ngoại, các ràng buộc toàn vẹn. Mọi thứ đều có cấu trúc, có thể dự đoán và có thể truy vấn chính xác bằng SQL. Đây là mô hình tổ chức dữ liệu đã được kiểm chứng qua hàng thập kỷ phát triển phần mềm.

Bây giờ hãy hình dung bạn cần xây dựng một hệ thống có thể trả lời câu hỏi: "Chính sách bảo hành của công ty đối với sản phẩm điện tử là gì?" Dữ liệu để trả lời câu hỏi này có thể nằm rải rác trong hàng trăm tài liệu PDF, email, trang web nội bộ và slide thuyết trình – những dữ liệu không có cấu trúc cố định, không được chuẩn hóa và không thể truy vấn bằng SQL. Đây chính là thách thức mà tổ chức dữ liệu trong ứng dụng AI tạo sinh phải giải quyết.

Sự khác biệt không chỉ nằm ở loại dữ liệu. Trong ứng dụng truyền thống, bạn viết mã để xử lý dữ liệu theo các quy tắc nghiệp vụ xác định. Trong ứng dụng AI tạo sinh, một phần đáng kể "logic xử lý" được ủy thác cho mô hình AI thông qua các lời gọi API – mô hình không chỉ truy xuất dữ liệu mà còn diễn giải, tổng hợp và sinh ra nội dung mới từ dữ liệu đó. Điều này đòi hỏi một cách tiếp cận hoàn toàn khác về cách chúng ta tổ chức, lưu trữ và truy xuất dữ liệu.

### *7.1.2 Sự khác biệt căn bản giữa tổ chức dữ liệu truyền thống và GenAI*

Trước khi đi vào các nguyên tắc thiết kế cụ thể, cần hiểu rõ những khác biệt nền tảng trong cách tổ chức dữ liệu giữa hai mô hình ứng dụng. Bảng 7.1 tóm tắt những khác biệt cốt lõi này.

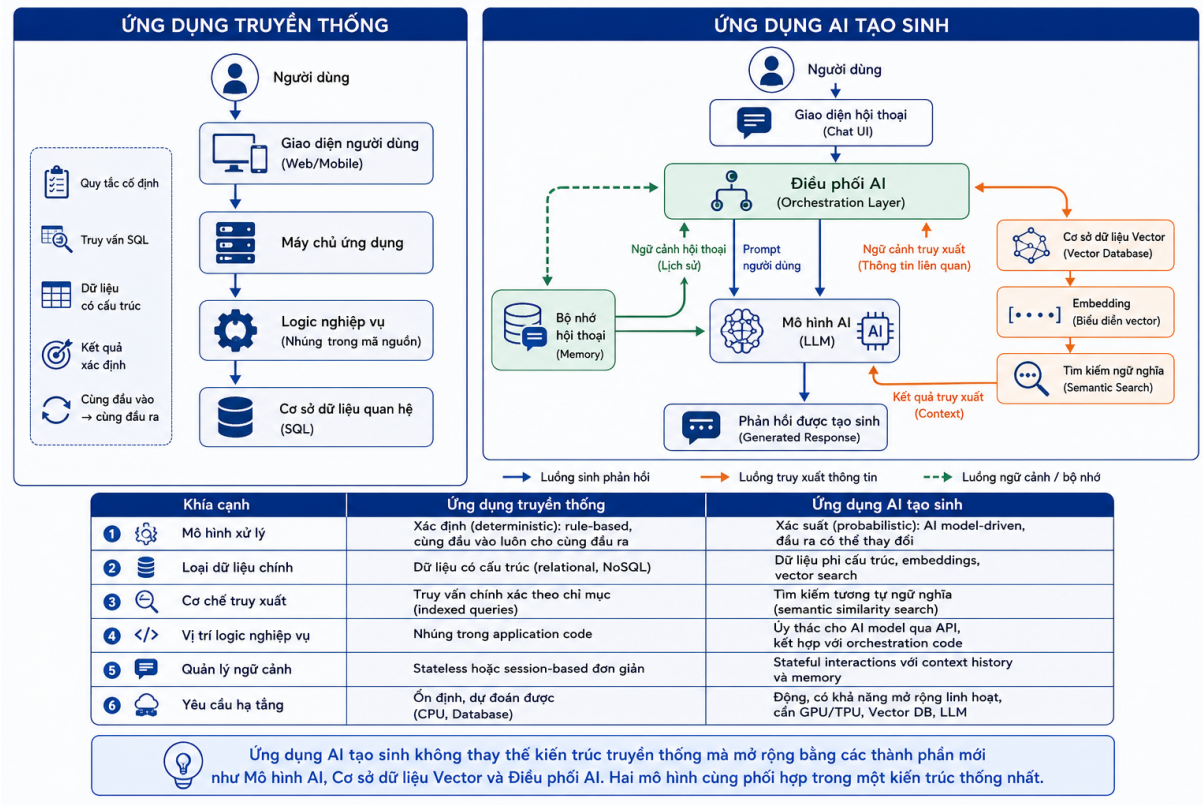
| **Khía cạnh** | **Ứng dụng truyền thống** | **Ứng dụng AI tạo sinh** |
|---|---|---|
| Mô hình xử lý | Xác định (deterministic): rule-based, cùng đầu vào luôn cho cùng đầu ra | Xác suất (probabilistic): AI model-driven, đầu ra có thể thay đổi |
| Loại dữ liệu chính | Dữ liệu có cấu trúc (relational, NoSQL) | Dữ liệu phi cấu trúc, embeddings, vector search |
| Cơ chế truy xuất | Truy vấn chính xác theo chỉ mục (indexed queries) | Tìm kiếm tương tự ngữ nghĩa (semantic similarity search) |
| Vị trí logic nghiệp vụ | Nhúng trong application code | Ủy thác cho AI model qua API, kết hợp với orchestration code |
| Quản lý ngữ cảnh | Stateless hoặc session-based đơn giản | Stateful interactions với context history và memory |
| Yêu cầu hạ tầng | Ổn định, dự đoán được | Động, có khả năng mở rộng linh hoạt, cần GPU/TPU |

**Bảng 7.1:** So sánh tổ chức dữ liệu: ứng dụng truyền thống và ứng dụng AI tạo sinh

Những khác biệt này không có nghĩa là các nguyên tắc tổ chức dữ liệu truyền thống trở nên lỗi thời. Trong thực tế, một ứng dụng AI tạo sinh thường kết hợp cả hai mô hình: cơ sở dữ liệu quan hệ vẫn được dùng để quản lý người dùng, đơn hàng và các dữ liệu nghiệp vụ có cấu trúc; vector database được bổ sung để xử lý tìm kiếm ngữ nghĩa và truy xuất tri thức. Chìa khóa nằm ở việc hiểu khi nào dùng công cụ nào và cách chúng phối hợp với nhau trong một kiến trúc thống nhất.

### *7.1.3 Bảy nguyên tắc thiết kế kiến trúc dữ liệu cho GenAI*

Khi thiết kế tổ chức dữ liệu cho ứng dụng AI tạo sinh, bảy nguyên tắc sau đây đóng vai trò định hướng cho mọi quyết định kiến trúc. Đây không phải là một danh sách kiểm tra cứng nhắc, mà là một khung tư duy giúp bạn cân nhắc các yếu tố then chốt trong từng bối cảnh cụ thể.




| Khía cạnh | Ứng dụng truyền thống | Ứng dụng AI tạo sinh |
|---|---|---|
| 1 Mô hình xử lý | Xác định (deterministic): rule-based, cùng đầu vào luôn cho cùng đầu ra | Xác suất (probabilistic): AI model-driven, đầu ra có thể thay đổi |
| 2 Loại dữ liệu chính | Dữ liệu có cấu trúc (relational, NoSQL) | Dữ liệu phi cấu trúc, embeddings, vector search |
| 3 Cơ chế truy xuất | Truy vấn chính xác theo chỉ mục (indexed queries) | Tìm kiếm tương tự ngữ nghĩa (semantic similarity search) |
| 4 Vị trí logic nghiệp vụ | Nhúng trong application code | Ủy thác cho AI model qua API, kết hợp với orchestration code |
| 5 Quản lý ngữ cảnh | Stateless hoặc session-based đơn giản | Stateful interactions với context history và memory |
| 6 Yêu cầu hạ tầng | Ổn định, dự đoán được (CPU, Database) | Động, có khả năng mở rộng linh hoạt, cần GPU/TPU, Vector DB, LLM |

Ứng dụng AI tạo sinh không thay thế kiến trúc truyền thống mà mở rộng bằng các thành phần mới như Mô hình AI, Cơ sở dữ liệu Vector và Điều phối AI. Hai mô hình cùng phối hợp trong một kiến trúc thống nhất.

**Hình 7.1:** So sánh kiến trúc tổng thể: ứng dụng truyền thống (trái) và ứng dụng AI tạo sinh (phải). Bên phải thể hiện rõ sự xuất hiện của AI Model, Vector DB và Orchestration layer – những thành phần không có trong kiến trúc truyền thống.

### *a. Nguyên tắc 1: Thiết kế module hóa và hướng API*

Trong kiến trúc GenAI, mỗi thành phần – từ data retrieval, processing, inference đến storage – nên được thiết kế như một module độc lập, giao tiếp qua API được định nghĩa rõ ràng. Cách tiếp cận này mang lại ba lợi ích chính.

Thứ nhất, nó cho phép mỗi module được phát triển, kiểm thử và mở rộng (scale) một cách độc lập. Khi lượng truy vấn tăng đột biến, bạn có thể scale riêng tầng inference mà không cần thay đổi tầng dữ liệu. Thứ hai, nó cô lập sự phụ thuộc vào các công nghệ cụ thể – nếu bạn muốn chuyển từ Pinecone sang Milvus cho vector database, chỉ module truy xuất dữ liệu bị ảnh hưởng. Thứ ba, nó tạo điều kiện cho việc thử nghiệm: bạn có thể A/B test hai mô hình embedding khác nhau hoặc hai chiến lược chunking khác nhau mà không làm gián đoạn toàn bộ hệ thống.

Một hệ quả quan trọng của nguyên tắc này là business logic không còn được nhúng trực tiếp trong application code như truyền thống, mà được ủy thác cho AI model thông qua các lời gọi API đến các dịch vụ inference như Amazon Bedrock, OpenAI, hay Claude. Mã nguồn ứng dụng chuyển sang vai trò điều phối (orchestration): quyết định khi nào gọi model nào, với dữ liệu gì, và xử lý kết quả ra sao.

### *b. Nguyên tắc 2: Xử lý dữ liệu có ngữ cảnh*

Ứng dụng GenAI không chỉ xử lý dữ liệu thô – nó phải hiểu và tận dụng ngữ cảnh (context) để tạo ra phản hồi phù hợp. Điều này có nghĩa là hệ thống cần có khả năng:

- Xác định ngữ cảnh liên quan đến truy vấn hiện tại từ kho dữ liệu khổng lồ.
- Kết hợp ngữ cảnh từ nhiều nguồn: lịch sử hội thoại, tài liệu tham khảo, dữ liệu người dùng, thông tin thời gian thực.
- Quản lý giới hạn context window của mô hình – nếu context quá dài, thông tin quan trọng có thể bị cắt bỏ; nếu quá ngắn, mô hình thiếu thông tin để trả lời chính xác.

Cơ chế chính để hiện thực hóa nguyên tắc này là RAG (Retrieval-Augmented Generation): thay vì nhồi nhét toàn bộ tài liệu vào prompt, hệ thống chỉ truy xuất những đoạn thực sự liên quan đến câu hỏi và đưa chúng vào ngữ cảnh trước khi gọi LLM. Chúng ta sẽ đi sâu vào RAG trong Mục 7.2.

### *c. Nguyên tắc 3: Suy luận mô hình có khả năng mở rộng và hiệu quả*

Model inference – quá trình mô hình AI sinh ra kết quả từ đầu vào – là điểm tiêu tốn tài nguyên nhất trong toàn bộ hệ thống. Nguyên tắc này đòi hỏi kiến trúc phải được thiết kế để inference có thể mở rộng theo tải và vận hành với chi phí hợp lý.

Các chiến lược cụ thể bao gồm: sử dụng batch processing để gom nhóm các yêu cầu không khẩn cấp; triển khai các tầng caching để tránh gọi lại model cho các truy vấn tương tự; áp dụng model quantization và distillation để giảm kích thước mô hình mà không mất nhiều chất lượng; và sử dụng GPU acceleration để tăng tốc độ suy luận.

### *d. Nguyên tắc 4: Học liên tục và thích ứng mô hình*

Không giống như ứng dụng truyền thống nơi logic chỉ thay đổi khi có bản cập nhật mã nguồn, ứng dụng GenAI cần cơ chế để mô hình liên tục được cập nhật, fine-tune và điều chỉnh cho phù hợp với mục tiêu kinh doanh. Điều này đòi hỏi:

- Hệ thống thu thập phản hồi từ người dùng (thumbs up/down, đánh giá chất lượng).
- Pipeline đánh giá định kỳ chất lượng đầu ra của mô hình.
- Cơ chế MLOps để quản lý vòng đời mô hình: từ huấn luyện, đánh giá, triển khai đến giám sát.
- Khả năng cập nhật cơ sở tri thức (document store, vector database) mà không cần triển khai lại toàn bộ hệ thống.

#### *e. Nguyên tắc 5: Bảo mật, tuân thủ và AI có đạo đức*

Dữ liệu trong ứng dụng GenAI thường bao gồm cả dữ liệu nhạy cảm của người dùng và tài liệu nội bộ của tổ chức. Nguyên tắc này yêu cầu:

- Mã hóa dữ liệu cả khi truyền (in transit) và khi lưu trữ (at rest).
- Kiểm soát truy cập theo nguyên tắc đặc quyền tối thiểu (least privilege).
- Cơ chế lọc nội dung (content moderation) để ngăn chặn đầu ra độc hại hoặc không phù hợp.
- Tuân thủ các quy định về quyền riêng tư dữ liệu (GDPR, các quy định của Việt Nam về bảo vệ dữ liệu cá nhân).
- Audit logging để theo dõi mọi truy cập và xử lý dữ liệu.

#### *f. Nguyên tắc 6: Triển khai linh hoạt và tương thích đa nền tảng*

Ứng dụng GenAI nên được thiết kế để có thể vận hành trên nhiều môi trường khác nhau – từ cloud (AWS, GCP, Azure) đến on-premise, từ managed services đến self-hosted infrastructure. Tính linh hoạt này đặc biệt quan trọng khi:

- Tổ chức có yêu cầu về chủ quyền dữ liệu (data sovereignty), buộc dữ liệu phải nằm trong một khu vực địa lý cụ thể.
- Chi phí của managed services tăng theo quy mô và đến một ngưỡng nào đó, self-hosting trở nên kinh tế hơn.
- Cần tránh phụ thuộc vào một nhà cung cấp duy nhất (vendor lock-in).

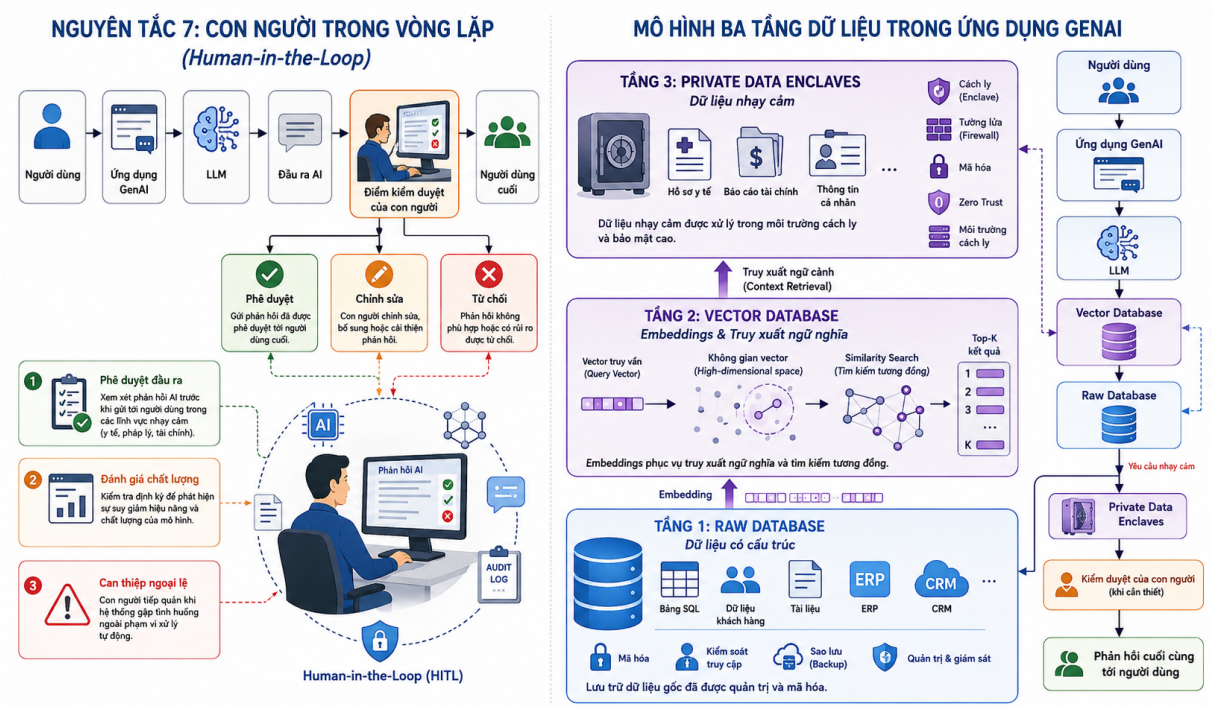
#### *g. Nguyên tắc 7: Con người trong vòng lặp*

Human-in-the-loop (HITL) là nguyên tắc thiết kế các điểm kiểm soát nơi con người có thể can thiệp vào quyết định của AI. Điều này đặc biệt quan trọng đối với các ứng dụng có tác động lớn đến người dùng hoặc doanh nghiệp.

Các điểm HITL điển hình bao gồm: xem xét và phê duyệt đầu ra trước khi gửi đến người dùng cuối (đối với các ứng dụng nhạy cảm như tư vấn pháp lý, y tế); đánh giá chất lượng định kỳ để phát hiện sự suy giảm hiệu năng của mô hình; và can thiệp khi hệ thống gặp tình huống nằm ngoài khả năng xử lý tự động.

### *7.1.4 Mô hình ba tầng dữ liệu trong ứng dụng GenAI*

Một trong những đóng góp quan trọng nhất của kiến trúc GenAI cho tổ chức dữ liệu là mô hình ba tầng (three-tier data model). Mô hình này phân tách dữ liệu thành ba tầng với vai trò và yêu cầu bảo mật khác nhau.




**Hình 7.2:** Mô hình ba tầng dữ liệu trong ứng dụng GenAI: Raw Database (dữ liệu có cấu trúc, mã hóa), Vector Database (embeddings, similarity search), và Private Data Enclaves (dữ liệu nhạy cảm, môi trường cách ly).

### *a. Tầng 1: Cơ sở dữ liệu thô (Raw Database)*

Đây là tầng lưu trữ dữ liệu người dùng và dữ liệu nghiệp vụ cốt lõi của ứng dụng, như thông tin tài khoản, lịch sử giao dịch, hồ sơ khách hàng. Yêu cầu chính của tầng này là:

- Lưu trữ an toàn với mã hóa cả khi truyền (in transit) và khi lưu trữ (at rest).
- Sao lưu định kỳ và khả năng phục hồi sau sự cố (disaster recovery).
- Kiểm soát truy cập nghiêm ngặt theo vai trò.
- Tuân thủ các quy định về bảo vệ dữ liệu.

Tầng này thường sử dụng các cơ sở dữ liệu quan hệ truyền thống (PostgreSQL, MySQL) hoặc NoSQL (MongoDB, DynamoDB) – những công nghệ đã được kiểm chứng về độ tin cậy và bảo mật.

### *b. Tầng 2: Cơ sở dữ liệu vector (Vector Database)*

Đây là tầng đặc trưng nhất của ứng dụng GenAI, lưu trữ các vector embedding – biểu diễn số học của ngữ nghĩa văn bản – phục vụ cho việc tìm kiếm tương tự và truy xuất tri thức nhanh chóng. Khác với tầng Raw Database lưu trữ dữ liệu ở dạng "con người đọc được", tầng này lưu trữ dữ liệu ở dạng "máy học hiểu được".

Vector database cho phép thực hiện các truy vấn như: "tìm 5 đoạn văn bản có ý nghĩa gần nhất với câu hỏi này", với thời gian phản hồi ở mức mili-giây ngay cả trên tập dữ liệu hàng triệu vector. Các lựa chọn phổ biến bao gồm Pinecone, Weaviate, Milvus, Qdrant, Chroma và pgvector (một extension của PostgreSQL).

#### *c. Tầng 3: Vùng dữ liệu riêng tư (Private Data Enclaves)*

Đây là tầng cách ly dành cho dữ liệu đặc biệt nhạy cảm mà tổ chức không muốn – hoặc không được phép – đưa vào các pipeline xử lý AI thông thường. Ví dụ: dữ liệu tài chính của khách hàng VIP, hồ sơ y tế, hoặc thông tin thuộc diện bí mật kinh doanh.

Tầng này hoạt động theo nguyên tắc: dữ liệu chỉ được truy cập trong các môi trường biệt lập, được kiểm soát chặt chẽ, và không bao giờ rời khỏi phạm vi kiểm soát của tổ chức. Các mô hình AI xử lý dữ liệu trong tầng này thường được triển khai on-premise hoặc trong private cloud, thay vì gọi API bên ngoài.

### *7.1.5 Tổ chức kho tri thức cho RAG*

Kho tri thức (knowledge store) là nơi lưu trữ toàn bộ tài liệu, thông tin và dữ liệu mà ứng dụng GenAI cần để trả lời câu hỏi của người dùng. Tổ chức kho tri thức hiệu quả là yếu tố quyết định chất lượng của hệ thống RAG.

Một kho tri thức đầy đủ thường bao gồm bốn loại lưu trữ:

1. **Vector Database**: lưu trữ vector embeddings và hỗ trợ similarity search. Đây là thành phần cốt lõi cho việc truy xuất ngữ nghĩa – tìm kiếm thông tin dựa trên ý nghĩa thay vì từ khóa chính xác.
2. **SQL Database**: lưu trữ metadata có cấu trúc như thông tin tác giả, ngày tạo, danh mục tài liệu. Những dữ liệu này hỗ trợ lọc và tinh chỉnh kết quả truy xuất.
3. **NoSQL Database**: lưu trữ dữ liệu phi cấu trúc hoặc bán cấu trúc như nội dung thô của tài liệu, JSON responses từ API.
4. **Object Storage**: lưu trữ các file dữ liệu thô như PDF, hình ảnh, video. Các dịch vụ như Amazon S3, Google Cloud Storage thường được sử dụng cho tầng này.

### *7.1.6 Chiến lược chia nhỏ dữ liệu phi cấu trúc*

Trước khi dữ liệu phi cấu trúc (tài liệu, văn bản) có thể được lưu trữ và truy xuất trong vector database, chúng cần được chia nhỏ thành các đoạn có ý nghĩa – một quá trình gọi là chunking. Việc lựa chọn chiến lược chunking ảnh hưởng trực tiếp đến chất lượng truy xuất: chunk quá lớn có thể chứa nhiều thông tin không liên quan, làm nhiễu kết quả; chunk quá nhỏ có thể mất ngữ cảnh, khiến mô hình không đủ thông tin để trả lời.

Ba chiến lược chunking chính được sử dụng phổ biến:

- **Semantic Chunking (chia theo ngữ nghĩa)**: chia tài liệu tại các ranh giới tự nhiên về mặt ý nghĩa – kết thúc một chủ đề, bắt đầu một chủ đề mới. Chiến lược này cho chất lượng truy xuất tốt nhất vì mỗi chunk chứa một ý tưởng hoàn chỉnh, nhưng đòi hỏi mô hình hoặc thuật toán có khả năng phát hiện ranh giới ngữ nghĩa.
- **Recursive Chunking (chia phân cấp)**: chia tài liệu theo cấu trúc phân cấp sẵn có – chương, mục, tiểu mục. Chiến lược này bảo toàn cấu trúc logic của tài liệu, phù hợp với các tài liệu kỹ thuật, giáo trình, sách có cấu trúc rõ ràng.
- **Paragraph Chunking (chia theo đoạn văn)**: chia tài liệu theo ranh giới đoạn văn tự nhiên. Đây là chiến lược đơn giản nhất, dễ triển khai và thường cho kết quả tốt với các tài liệu được viết có cấu trúc đoạn văn rõ ràng.

Sau khi chia nhỏ, mỗi chunk được chuyển qua một embedding model để biến đổi thành một vector số (numerical vector) đại diện cho ngữ nghĩa của nó. Tập hợp các

vector này được lưu vào vector database, sẵn sàng cho việc truy xuất khi có câu hỏi từ người dùng.

### *7.1.7 Hạ tầng cho ứng dụng AI tạo sinh*

Tổ chức dữ liệu không thể tách rời khỏi hạ tầng vận hành bên dưới. Ứng dụng GenAI đòi hỏi hạ tầng năng động hơn, có khả năng mở rộng hơn và tiêu tốn nhiều tài nguyên hơn so với ứng dụng truyền thống. Ba miền hạ tầng cốt lõi cần được quan tâm:

1. **Tài nguyên tính toán (Computing Resources)**: GPU và TPU cho model inference và training; khả năng mở rộng theo nhu cầu (on-demand scaling); môi trường tính toán hiệu năng cao cho các tác vụ xử lý dữ liệu lớn như embedding generation hàng loạt.
2. **Tích hợp hệ thống (System Integration)**: pipeline dữ liệu có độ trễ thấp để truyền dữ liệu giữa các thành phần; điều phối khối lượng công việc (workload orchestration) hiệu quả; API gateway để quản lý và bảo vệ các điểm cuối (endpoint).
3. **Quản trị (Governance)**: giám sát và cảnh báo (monitoring và alerting) cho toàn bộ hệ thống; theo dõi chi phí token và tài nguyên; quản lý phiên bản mô hình và dữ liệu; đảm bảo tuân thủ các chính sách bảo mật và quy định.

### *7.1.8 Từ tổ chức dữ liệu đến truy xuất thông minh*

Mô hình tổ chức dữ liệu ba tầng và các nguyên tắc thiết kế được trình bày trong mục này tạo nền tảng cho một câu hỏi then chốt: làm thế nào để ứng dụng có thể truy xuất chính xác thông tin cần thiết từ kho dữ liệu khổng lồ và biến nó thành câu trả lời có ý nghĩa cho người dùng? Đây chính là bài toán mà mô hình RAG (Retrieval-Augmented Generation) giải quyết.

Trước khi chuyển sang chủ đề đó, hãy tự kiểm tra: Bạn có thể giải thích vì sao vector database không thay thế relational database, mà bổ sung cho nó trong kiến trúc GenAI? Và trong mô hình ba tầng dữ liệu, điều gì sẽ xảy ra nếu dữ liệu nhạy cảm từ Private Data Enclave vô tình bị đưa vào Vector Database?

Mục tiếp theo sẽ đi sâu vào kiến trúc RAG – cầu nối giữa tổ chức dữ liệu và khả năng sinh nội dung thông minh của ứng dụng AI tạo sinh.

## 7.2 Mô hình RAG và cơ sở dữ liệu vector

*Sau khi hoàn thành mục này, sinh viên có thể:*

- *Giải thích được khái niệm, cơ chế hoạt động và lợi ích của mô hình RAG (Retrieval-Augmented Generation).*
- *So sánh được ba mức độ phát triển của ứng dụng hỏi đáp: simple prompt, context-augmented và RAG-based.*
- *Phân biệt được Traditional RAG và Agentic RAG qua bốn đặc điểm khác biệt chính.*
- *Mô tả được kiến trúc sáu bước của Agentic RAG và vai trò của từng bước.*
- *Trình bày được các loại cơ sở dữ liệu vector phổ biến và tiêu chí lựa chọn.*

### 7.2.1 Khởi động: Khi mô hình ngôn ngữ không đủ

Hãy thực hiện một thí nghiệm tư duy. Bạn hỏi ChatGPT: "Chính sách nghỉ phép năm 2026 của công ty tôi là gì?" Câu trả lời bạn nhận được – dù có tự tin đến đâu – cũng chỉ là sự suy đoán dựa trên dữ liệu huấn luyện của mô hình, vốn không hề chứa chính sách nội bộ của công ty bạn. Nếu may mắn, mô hình sẽ thừa nhận rằng nó không biết. Nếu không, nó có thể "bịa ra" một chính sách nghe có vẻ hợp lý nhưng hoàn toàn sai – một hiện tượng mà chúng ta gọi là hallucination (ảo giác).

Vấn đề cốt lõi ở đây là: LLM chỉ có kiến thức được "đóng băng" tại thời điểm huấn luyện. Nó không biết những gì đã thay đổi sau đó, không truy cập được vào dữ liệu nội bộ của tổ chức bạn, và không có cách nào để phân biệt giữa "tôi biết điều này" và "tôi đang bịa ra điều này".

Mô hình RAG (Retrieval-Augmented Generation – Sinh nội dung tăng cường truy xuất) ra đời để giải quyết chính xác vấn đề này. Thay vì yêu cầu mô hình trả lời chỉ dựa trên kiến thức nội tại, RAG cho phép mô hình "tra cứu" thông tin từ các nguồn dữ liệu bên ngoài trước khi sinh câu trả lời – giống như một sinh viên được phép mở sách trong phòng thi vậy.

### 7.2.2 Nguyên lý hoạt động của RAG

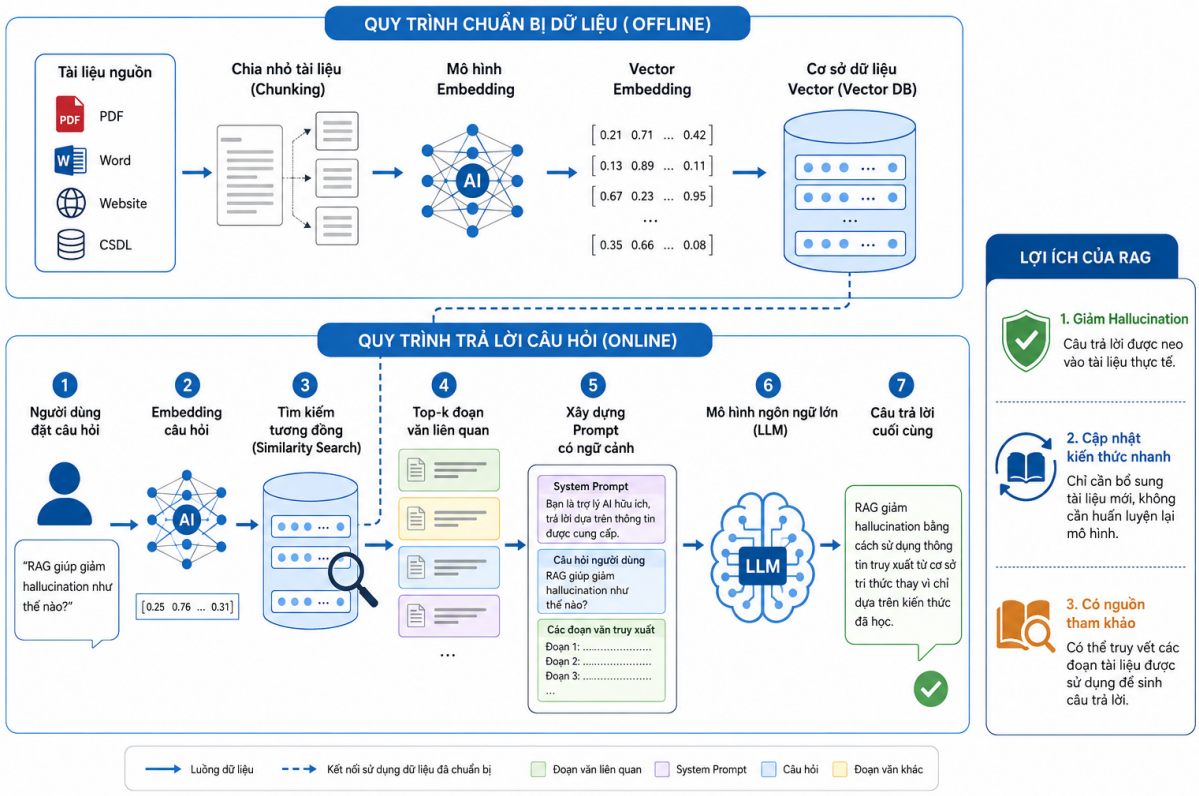
Về bản chất, RAG là sự kết hợp giữa hai thành phần: retrieval (truy xuất) và generation (sinh nội dung). Quy trình hoạt động của RAG có thể được mô tả qua các bước sau:

1. **Người dùng đặt câu hỏi:** Câu hỏi được gửi đến hệ thống RAG.
2. **Chuyển đổi câu hỏi thành embedding:** Câu hỏi được đưa qua một embedding model để tạo ra một vector đại diện cho ngữ nghĩa của nó.
3. **Truy xuất ngữ cảnh liên quan:** Vector của câu hỏi được sử dụng để tìm kiếm trong vector database. Hệ thống trả về top-k đoạn văn bản (chunks) có ngữ nghĩa gần nhất với câu hỏi.
4. **Xây dựng prompt có ngữ cảnh:** Các đoạn văn bản truy xuất được kết hợp với câu hỏi gốc (và thường kèm theo system prompt) để tạo thành prompt hoàn chỉnh gửi đến LLM.
5. **LLM sinh câu trả lời:** LLM xử lý prompt đã được làm giàu ngữ cảnh và sinh ra câu trả lời cuối cùng – lần này đã được "tiếp đất" (grounded) bởi thông tin thực tế từ cơ sở tri thức.

Điều kỳ diệu của RAG nằm ở chỗ nó giải quyết đồng thời ba vấn đề lớn của LLM: (1) **giảm hallucination** – vì câu trả lời được neo vào thông tin thực tế từ cơ sở tri thức; (2) **cập nhật kiến thức không cần huấn luyện lại** – chỉ cần cập nhật tài liệu trong cơ sở tri thức, mô hình sẽ tự động "biết" thông tin mới; và (3) **cung cấp nguồn tham khảo** – người dùng có thể kiểm tra xem câu trả lời đến từ tài liệu nào, tăng độ tin cậy của hệ thống.

### 7.2.3 Ba mức độ phát triển của ứng dụng hỏi đáp

Để hiểu rõ hơn giá trị của RAG, hãy xem xét ba mức độ phát triển của một ứng dụng hỏi đáp (Q&A application), từ đơn giản đến phức tạp.




**Hình 7.3:** Kiến trúc RAG (Retrieval-Augmented Generation). Phía trên: quy trình offline – tài liệu được chunking, embedding và lưu vào Vector DB. Phía dưới: quy trình online – câu hỏi được embedding, tìm kiếm similarity trong Vector DB, và LLM sinh câu trả lời dựa trên các chunks truy xuất được.

#### a. *Mức 1: Hỏi đáp với prompt đơn giản*

Ở mức cơ bản nhất, ứng dụng chỉ đơn giản chuyển tiếp câu hỏi của người dùng đến LLM và trả về câu trả lời. Prompt có thể chỉ là: "`Trảlờicâuhỏisau:\{cau\_hoi\}`".

Hạn chế của cách tiếp cận này là rõ ràng: LLM chỉ có thể trả lời dựa trên kiến thức đã được huấn luyện. Mọi câu hỏi về thông tin nội bộ, dữ liệu mới cập nhật hoặc kiến thức chuyên ngành hẹp đều có nguy cơ nhận được câu trả lời sai hoặc bịa đặt. Đây là cách tiếp cận phù hợp cho các câu hỏi kiến thức phổ thông, nhưng không đủ cho các ứng dụng doanh nghiệp.

#### b. *Mức 2: Hỏi đáp có ngữ cảnh tĩnh*

Ở mức này, ứng dụng bổ sung một đoạn ngữ cảnh cố định vào prompt trước khi gửi đến LLM. Ví dụ, một chatbot chăm sóc khách hàng có thể luôn kèm theo đoạn mô tả về sản phẩm và chính sách chung của công ty trong mỗi prompt.

Cách tiếp cận này cải thiện chất lượng câu trả lời cho các câu hỏi phổ biến, nhưng vẫn có hai hạn chế lớn. Thứ nhất, ngữ cảnh là cố định – nếu người dùng hỏi về một sản phẩm cụ thể không có trong mô tả chung, hệ thống vẫn không thể trả lời. Thứ hai, context window có giới hạn – bạn không thể nhồi nhét toàn bộ kho tài liệu của công ty vào mỗi prompt.

#### c. *Mức 3: Hỏi đáp với RAG*

Ở mức này, ứng dụng không còn dùng ngữ cảnh cố định. Thay vào đó, với mỗi câu hỏi, hệ thống tự động truy xuất những đoạn thông tin liên quan nhất từ cơ sở tri thức và chỉ đưa những đoạn đó vào prompt. Điều này giải quyết được cả hai hạn chế của mức 2: ngữ cảnh luôn được cá nhân hóa cho từng câu hỏi, và chỉ những thông tin thực sự liên quan mới chiếm dung lượng trong context window.

Bảng 7.2 tóm tắt sự khác biệt giữa ba mức độ.

| **Khía cạnh** | **Mức 1: Simple Prompt** | **Mức 2: Context-Augmented** | **Mức 3: RAG** |
|---|---|---|---|
| Nguồn kiến thức | Chỉ từ huấn luyện LLM | Huấn luyện + ngữ cảnh tĩnh | Huấn luyện + truy xuất động |
| Khả năng cập nhật | Cần fine-tune hoặc train lại | Sửa ngữ cảnh thủ công | Cập nhật cơ sở tri thức là đủ |
| Độ chính xác với dữ liệu nội bộ | Rất thấp (dễ hallucination) | Trung bình (chỉ tốt với câu hỏi phổ biến) | Cao (luôn có ngữ cảnh phù hợp) |
| Sử dụng context window | Tối thiểu | Lãng phí cho ngữ cảnh không liên quan | Tối ưu – chỉ đưa vào thông tin liên quan |
| Khả năng trích dẫn nguồn | Không | Hạn chế | Có thể truy xuất nguồn gốc thông tin |

**Bảng 7.2:** So sánh ba mức độ phát triển của ứng dụng hỏi đáp

### 7.2.4 *Traditional RAG và Agentic RAG*

Khi các ứng dụng GenAI ngày càng phức tạp, mô hình RAG cũng phát triển từ dạng truyền thống (Traditional RAG) sang dạng tác tử (Agentic RAG). Sự khác biệt giữa hai mô hình này không chỉ là về mặt kỹ thuật mà còn phản ánh một sự thay đổi về triết lý

thiết kế.

#### *a. Traditional RAG*

Trong Traditional RAG, luồng xử lý được định nghĩa trước và cố định: nhận câu hỏi → tạo embedding → truy xuất vector → xây dựng prompt → gọi LLM → trả kết quả. Mỗi bước được thực thi theo một trình tự cứng nhắc, không có khả năng thích ứng với độ phức tạp của câu hỏi hay chất lượng của kết quả truy xuất.

Traditional RAG hoạt động tốt với các ứng dụng hỏi đáp đơn giản, nơi câu hỏi rõ ràng và câu trả lời có thể được tìm thấy trực tiếp trong tài liệu. Tuy nhiên, nó gặp khó khăn khi đối mặt với các câu hỏi phức tạp đòi hỏi suy luận nhiều bước, so sánh thông tin từ nhiều nguồn, hoặc cần gọi thêm các công cụ bên ngoài.

#### *b. Agentic RAG*

Agentic RAG thay thế luồng xử lý cố định bằng một AI agent có khả năng ra quyết định động. Thay vì tuân theo một kịch bản cứng nhắc, agent có thể: đánh giá chất lượng của kết quả truy xuất ban đầu và quyết định có cần truy vấn lại với cách tiếp cận khác không; chọn công cụ phù hợp (vector search, web search, SQL query, API call) dựa trên bản chất của câu hỏi; thực hiện suy luận nhiều bước với các kết quả trung gian; và học hỏi từ các tương tác trước đó để cải thiện chất lượng theo thời gian.

Bảng 7.3 tóm tắt bốn điểm khác biệt chính giữa hai mô hình.

| **Khía cạnh** | **Traditional RAG** | **Agentic RAG** |
|---|---|---|
| Cơ chế ra quyết định | Luồng cố định (fixed flow) | AI agent tự quyết định từng bước |
| Tích hợp công cụ | Chỉ dùng vector database | Đa công cụ động: APIs, vector DBs, web search, code execution |
| Khả năng thích ứng | Không thích ứng với chất lượng truy xuất | Thích ứng thời gian thực: truy vấn lại, tinh chỉnh, tổng hợp động |
| Mức độ tự chủ | Thực thi theo kịch bản | Tự điều khiển luồng công việc (self-driven), ra quyết định ở mức chuyên gia |

**Bảng 7.3:** So sánh Traditional RAG và Agentic RAG

### *7.2.5 Kiến trúc sáu bước của Agentic RAG*

Để hiểu rõ hơn cách Agentic RAG vận hành, hãy xem xét kiến trúc sáu bước của nó. Lưu ý rằng đây không phải là một pipeline tuyến tính cứng nhắc – agent có thể quay lại các bước trước đó, lặp lại hoặc bỏ qua một số bước tùy theo tình huống.

1. **Xử lý câu hỏi người dùng (User Query Processing):** Agent tiếp nhận và phân tích câu hỏi: đây là câu hỏi đơn giản hay phức tạp? Cần loại thông tin gì? Có cần làm rõ thêm không? Agent có thể đặt câu hỏi ngược lại để làm rõ yêu cầu trước khi tiếp tục.
2. **Truy xuất dữ liệu động (Dynamic Data Retrieval):** Dựa trên phân tích ở bước 1, agent quyết định truy xuất từ đâu: vector database cho tìm kiếm ngữ nghĩa, SQL database cho dữ liệu có cấu trúc, web search cho thông tin thời gian thực, hoặc API bên ngoài cho dữ liệu chuyên biệt. Agent có thể thực hiện nhiều truy

vấn song song đến các nguồn khác nhau.

3. **Xử lý tầng tri thức (Knowledge Layer Processing):** Kết quả truy xuất thô được xử lý: lọc nhiễu, loại bỏ trùng lặp, sắp xếp theo mức độ liên quan, và tổng hợp thành một biểu diễn tri thức có cấu trúc. Bước này có thể bao gồm reranking (sắp xếp lại kết quả theo tiêu chí khác) hoặc fusion (kết hợp kết quả từ nhiều nguồn).
4. **Ra quyết định tự động (Autonomous Decision-Making):** Agent đánh giá xem thông tin đã thu thập có đủ để trả lời chưa. Nếu chưa đủ, agent có thể quyết định: truy vấn lại với cách tiếp cận khác, yêu cầu người dùng cung cấp thêm thông tin, hoặc chia nhỏ câu hỏi thành các câu hỏi con và xử lý từng phần.
5. **Thực thi hành động (Action Execution):** Khi đã có đủ thông tin, agent thực hiện hành động cuối cùng: sinh câu trả lời qua LLM, gọi một API để thực hiện tác vụ (như đặt hàng, gửi email), hoặc tạo một báo cáo tổng hợp từ nhiều nguồn.
6. **Ghi nhớ và học hỏi (Memory & Learning):** Agent lưu lại kinh nghiệm từ tương tác này: câu hỏi nào đã được xử lý tốt? Chiến lược truy xuất nào hiệu quả? Thông tin nào hữu ích cho các câu hỏi tương tự trong tương lai? Bước này cho phép hệ thống cải thiện chất lượng theo thời gian.

### *7.2.6 Cơ sở dữ liệu vector – Trái tim của RAG*

Nếu RAG là bộ não của hệ thống truy xuất thông minh, thì vector database chính là trái tim bơm máu – nơi lưu trữ và cung cấp "nhiên liệu tri thức" cho toàn bộ quá trình.

#### *a. Vai trò và nguyên lý hoạt động*

Vector database là loại cơ sở dữ liệu được thiết kế đặc biệt để lưu trữ và tìm kiếm các vector embedding – những biểu diễn số học nhiều chiều của dữ liệu. Khác với cơ sở dữ liệu truyền thống tìm kiếm theo khớp chính xác (exact match), vector database tìm kiếm theo độ tương tự (similarity search) – tức là tìm những vector gần nhau nhất trong không gian nhiều chiều.

Hai độ đo khoảng cách phổ biến nhất được sử dụng trong vector database là:

- **Cosine Similarity**: đo góc giữa hai vector. Giá trị càng gần 1, hai vector càng "cùng hướng" về mặt ngữ nghĩa. Đây là độ đo phổ biến nhất cho tìm kiếm văn bản vì nó không bị ảnh hưởng bởi độ dài của vector.
- **Euclidean Distance**: đo khoảng cách hình học giữa hai điểm trong không gian. Khoảng cách càng nhỏ, hai vector càng gần nhau. Độ đo này phù hợp khi độ lớn của vector cũng mang thông tin có ý nghĩa.

#### *b. Các loại vector database phổ biến*

Thị trường vector database hiện nay rất đa dạng, từ các giải pháp managed service trên cloud đến các thư viện mã nguồn mở có thể tự triển khai. Bảng 7.4 so sánh một số lựa chọn phổ biến.

#### *c. Tiêu chí lựa chọn vector database*

Việc lựa chọn vector database phụ thuộc vào ngữ cảnh cụ thể của dự án. Các tiêu chí quan trọng cần cân nhắc bao gồm:

- **Quy mô dữ liệu**: bạn đang xử lý hàng nghìn hay hàng tỷ vector? Các giải pháp như Milvus được thiết kế cho quy mô cực lớn, trong khi Chroma hoặc pgvector

| Giải pháp | Loại | Thuật toán tìm kiếm | Điểm mạnh | Phù hợp cho |
|---|---|---|---|---|
| Pinecone | Managed cloud | Approximate Nearest Neighbor (ANN) | Không cần quản lý hạ tầng, scale tự động | Startup, dự án vừa và nhỏ cần triển khai nhanh |
| Weaviate | Managed / Self-hosted | HNSW (Hierar-chical Navigable Small World) | Tích hợp sẵn module AI, hybrid search (vector + keyword) | Ứng dụng cần tìm kiếm kết hợp |
| Milvus | Self-hosted (có cloud) | Nhiều thuật toán (IVF, HNSW, DiskANN) | Hiệu năng cao, scale đến hàng tỷ vector | Hệ thống quy mô lớn, yêu cầu hiệu năng cao |
| Chroma | Thư viện embed-ded / Server | ANN cơ bản | Đơn giản, dễ tích hợp, nhẹ | Prototype, dự án nhỏ, môi trường phát triển |
| pgvector | PostgreSQL extension | IVFFlat, HNSW | Tận dụng Post-greSQL có sẵn, một DB cho cả hai nhu cầu | Ứng dụng đã dùng Post-greSQL, cần giải pháp đơn giản |
| Qdrant | Self-hosted / Cloud | HNSW | Hiệu năng cao, API phong phú, lọc metadata mạnh | Ứng dụng cần filtering phức tạp trên metadata |

**Bảng 7.4:** So sánh một số giải pháp vector database phổ biến

phù hợp hơn cho quy mô khiêm tốn.

- **Yêu cầu về độ trễ**: ứng dụng của bạn cần phản hồi trong bao lâu? Các giải pháp managed service thường có độ trễ ổn định hơn, trong khi self-hosted có thể tối ưu cho phần cứng cụ thể.
- **Ngân sách và chi phí vận hành**: managed services tính phí theo usage (số lượng vector, số truy vấn), trong khi self-hosted đòi hỏi chi phí hạ tầng cố định và nhân lực vận hành.
- **Kỹ năng của đội ngũ**: đội ngũ của bạn có kinh nghiệm vận hành infrastructure không? Nếu không, managed service có thể là lựa chọn an toàn hơn.
- **Yêu cầu tích hợp**: bạn có cần tìm kiếm kết hợp vector + keyword (hybrid search) không? Có cần lọc metadata phức tạp không? Mỗi giải pháp có thế mạnh riêng về các tính năng này.
- **Yêu cầu về bảo mật và compliance**: dữ liệu có được phép rời khỏi hạ tầng của bạn không? Nếu không, self-hosted là lựa chọn bắt buộc.

### *7.2.7 Contextual Grounding – Đảm bảo câu trả lời đáng tin cậy*

Một thành phần quan trọng trong hệ thống RAG là cơ chế contextual grounding – đánh giá mức độ "bám sát" của câu trả lời vào thông tin nguồn. Một câu trả lời có thể nghe rất trôi chảy và thuyết phục, nhưng nếu nó không dựa trên các đoạn tài liệu đã truy xuất, đó có thể là hallucination.

Cơ chế contextual grounding hoạt động bằng cách so sánh từng khẳng định (claim) trong câu trả lời của LLM với nội dung của các đoạn tài liệu đã được truy xuất. Nếu một khẳng định không thể được xác minh từ nguồn, nó bị đánh dấu là "không có căn cứ" và có thể bị lọc bỏ hoặc yêu cầu sinh lại. Cơ chế này đặc biệt quan trọng trong các ứng dụng đòi hỏi độ chính xác cao như sinh mã nguồn, tư vấn pháp lý hoặc phân tích tài chính.

### *7.2.8 Các công cụ và framework triển khai RAG*

Để triển khai RAG trong thực tế, có nhiều framework và công cụ hỗ trợ. Ba cái tên nổi bật nhất hiện nay là:

- **LangChain**: framework mã nguồn mở cung cấp các abstraction cho toàn bộ pipeline RAG – từ document loading, chunking, embedding, vector store integration đến chain/tool execution. LangChain hỗ trợ cả Traditional RAG (qua chains) và Agentic RAG (qua agents với tool calling).
- **LlamaIndex**: framework chuyên sâu về indexing và retrieval, với nhiều chiến lược chunking, indexing và query engine được tối ưu sẵn. LlamaIndex đặc biệt mạnh trong việc xây dựng các cấu trúc index phức tạp như tree index, knowledge graph index.
- **Amazon Bedrock Knowledge Bases**: giải pháp managed service của AWS, cho phép tạo RAG application mà không cần quản lý hạ tầng vector database. Người dùng chỉ cần upload tài liệu lên S3, Bedrock tự động xử lý chunking, embedding, storage và retrieval.

### *7.2.9 Từ RAG đến Pipeline dữ liệu hoàn chỉnh*

Mô hình RAG và vector database cung cấp cơ chế truy xuất và sinh nội dung thông minh. Tuy nhiên, để toàn bộ hệ thống vận hành trơn tru, cần có một pipeline dữ liệu hoàn

chỉnh – từ khâu thu thập dữ liệu thô, xử lý, chuyển đổi thành embedding, đến lưu trữ và truy xuất. Đây chính là chủ đề của mục tiếp theo.

Trước khi chuyển sang, hãy suy ngẫm: Trong ba mức độ phát triển của ứng dụng hỏi đáp, mức nào phù hợp với dự án "Trợ lý phát triển phần mềm thông minh" của chúng ta? Liệu Agentic RAG có thực sự cần thiết cho mọi ứng dụng, hay đôi khi Traditional RAG đã là quá đủ?

## 7.3 Thiết kế pipeline dữ liệu cho AI tạo sinh

*Sau khi hoàn thành mục này, sinh viên có thể:*

- *Mô tả được cấu trúc tổng quát của một pipeline dữ liệu cho ứng dụng AI tạo sinh, bao gồm các giai đoạn và mối quan hệ giữa chúng.*
- *Thiết kế được pipeline dữ liệu cho một bài toán cụ thể, từ khâu thu thập đến sinh nội dung.*
- *Phân tích được sự khác biệt giữa pipeline ETL truyền thống và pipeline dữ liệu cho GenAI.*
- *Áp dụng được các kỹ thuật xử lý dữ liệu đặc thù như MapReduce summarization, xử lý đa phương thức và quản lý context window.*
- *Đánh giá được tầm quan trọng của chất lượng dữ liệu đầu vào đối với chất lượng đầu ra của hệ thống AI.*

### *7.3.1 Khởi động: Từ dữ liệu thô đến câu trả lời thông minh*

Hãy tưởng tượng bạn vừa được giao nhiệm vụ xây dựng hệ thống hỏi đáp cho toàn bộ tài liệu nội bộ của công ty – khoảng 10.000 tài liệu bao gồm PDF, Word, email, slide thuyết trình và trang web nội bộ. Bạn đã đọc về RAG và vector database ở mục trước, và bạn biết mình cần chuyển đổi tất cả những tài liệu này thành vector embeddings và lưu vào vector database.

Nhưng giữa "có tài liệu thô" và "có vector database sẵn sàng truy xuất" là một khoảng cách rất lớn. Làm thế nào để trích xuất văn bản từ PDF có chứa hình ảnh và bảng biểu? Làm thế nào để xử lý tài liệu tiếng Việt có dấu? Chia tài liệu thành các chunk với kích thước bao nhiêu là phù hợp? Khi tài liệu được cập nhật, làm thế nào để đồng bộ với vector database? Và làm thế nào để đảm bảo chất lượng dữ liệu đầu vào – bởi như người ta thường nói, "garbage in, garbage out"?

Pipeline dữ liệu (data pipeline) chính là câu trả lời cho tất cả những câu hỏi trên. Nó là "dây chuyền sản xuất" biến đổi dữ liệu thô, phi cấu trúc thành "nhiên liệu tri thức" có cấu trúc, sẵn sàng cho hệ thống AI truy xuất và sử dụng.

### *7.3.2 Pipeline dữ liệu cho GenAI khác gì so với ETL truyền thống?*

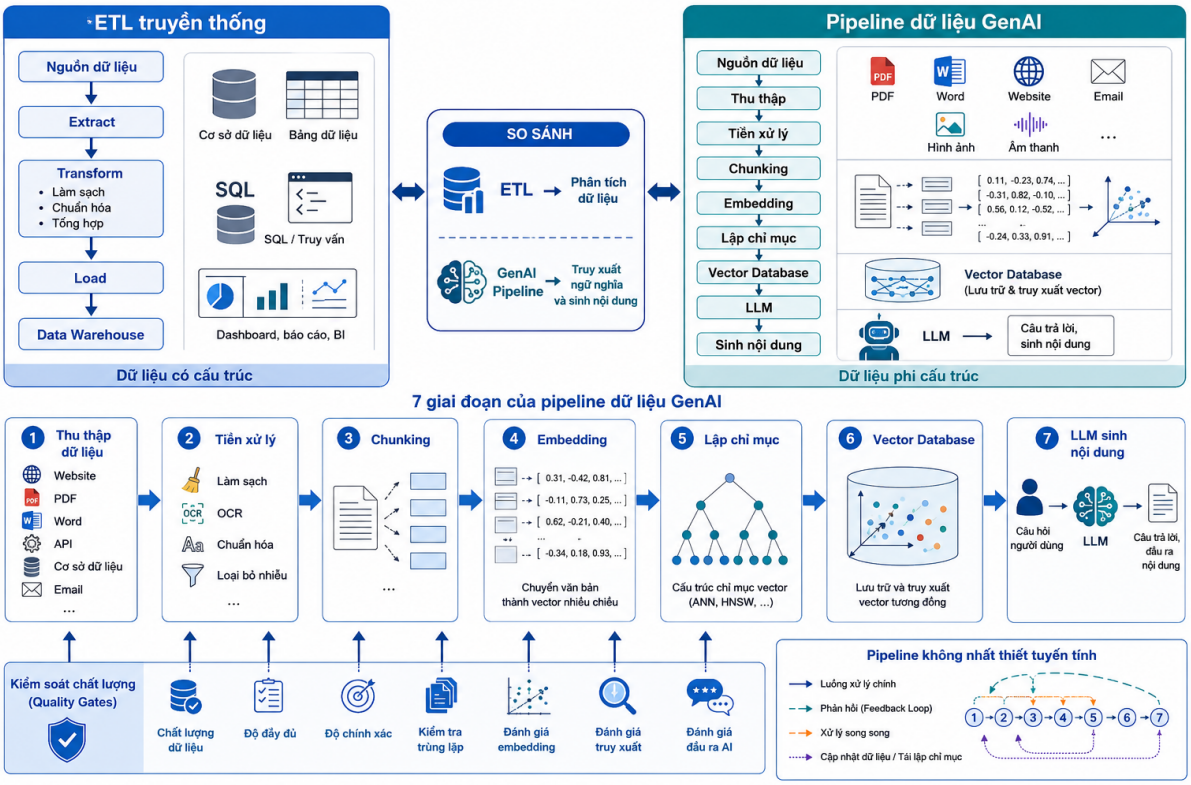
Trong phát triển phần mềm truyền thống, ETL (Extract – Transform – Load) là mô hình quen thuộc để xử lý dữ liệu: trích xuất dữ liệu từ nguồn, biến đổi theo quy tắc nghiệp vụ (làm sạch, chuẩn hóa, tổng hợp), và nạp vào kho dữ liệu (data warehouse). Mô hình này hoạt động tốt với dữ liệu có cấu trúc và quy tắc biến đổi xác định.

Pipeline dữ liệu cho GenAI mở rộng mô hình ETL theo những cách quan trọng. Thứ nhất, nó xử lý chủ yếu dữ liệu phi cấu trúc (văn bản tự do, hình ảnh, âm thanh)

thay vì các bản ghi có cấu trúc. Thứ hai, bước "Transform" không chỉ là làm sạch và chuẩn hóa mà còn bao gồm các thao tác đặc thù của AI: chunking (chia nhỏ văn bản), embedding (chuyển đổi thành vector số), và indexing (tạo chỉ mục cho tìm kiếm vector). Thứ ba, đích đến không phải là data warehouse để làm báo cáo, mà là vector database để phục vụ truy xuất ngữ nghĩa theo thời gian thực.

### 7.3.3 Cấu trúc tổng quát của pipeline dữ liệu GenAI

Một pipeline dữ liệu cho ứng dụng AI tạo sinh thường bao gồm bảy giai đoạn chính, từ thu thập dữ liệu thô đến sinh nội dung. Hình 7.4 minh họa toàn bộ pipeline cùng với các điểm kiểm soát chất lượng xuyên suốt. Lưu ý rằng trong thực tế, các giai đoạn này không nhất thiết phải là tuyến tính – một số có thể chạy song song, một số có thể được lặp lại, và luôn có các vòng phản hồi để cải thiện chất lượng.




**Hình 7.4:** Bảy giai đoạn của pipeline dữ liệu GenAI, từ thu thập dữ liệu thô đến sinh nội dung. Các mũi tên từ dưới lên thể hiện các điểm kiểm soát chất lượng (quality gates) xuyên suốt pipeline.

### 7.3.4 Giai đoạn 1–2: Thu thập và tiền xử lý dữ liệu

Giai đoạn đầu tiên của pipeline là thu thập dữ liệu từ các nguồn khác nhau. Trong bối cảnh GenAI, dữ liệu có thể đến từ rất nhiều nguồn đa dạng: tài liệu nội bộ (PDF, Word, Markdown), hình ảnh và biểu đồ, nội dung web (HTML, RSS), API bên ngoài (REST, GraphQL), luồng dữ liệu thời gian thực (Kafka, Kinesis), và cơ sở dữ liệu hiện có của tổ chức.

Sau khi thu thập, dữ liệu thô cần được tiền xử lý (preprocessing) để đảm bảo chất lượng đầu vào cho các bước sau. Các thao tác tiền xử lý điển hình bao gồm:

- **Trích xuất văn bản**: từ PDF (sử dụng các thư viện như PyMuPDF, pdfplumber), từ hình ảnh (OCR với Tesseract hoặc các dịch vụ cloud), từ HTML (loại bỏ thẻ, script, style).
- **Làm sạch dữ liệu**: loại bỏ ký tự đặc biệt không mong muốn, chuẩn hóa khoảng trắng, xử lý encoding (đặc biệt quan trọng với dữ liệu tiếng Việt có dấu).
- **Chuẩn hóa định dạng**: chuyển đổi tất cả về một định dạng văn bản thống nhất (thường là plain text hoặc Markdown) để các bước sau xử lý nhất quán.
- **Lọc và phân loại**: loại bỏ nội dung không liên quan hoặc trùng lặp, phân loại tài liệu theo chủ đề để tối ưu việc truy xuất sau này.
- **Trích xuất metadata**: thu thập thông tin đi kèm như tiêu đề, tác giả, ngày tạo, danh mục – những thông tin này rất hữu ích cho việc lọc và sắp xếp kết quả truy xuất.

Chất lượng của giai đoạn tiền xử lý ảnh hưởng trực tiếp đến toàn bộ pipeline. Một lỗi nhỏ trong việc trích xuất văn bản từ PDF (như mất dấu tiếng Việt, hoặc nhầm lẫn giữa các cột trong bảng) có thể lan truyền qua tất cả các bước sau và dẫn đến kết quả truy xuất sai lệch. Đầu tư thời gian cho bước này luôn xứng đáng.

### *7.3.5 Giai đoạn 3: Chiến lược chia nhỏ dữ liệu*

Sau khi có văn bản sạch, bước tiếp theo là chia nhỏ (chunking) thành các đoạn có kích thước phù hợp. Đây là một trong những quyết định thiết kế quan trọng nhất trong pipeline, vì nó ảnh hưởng trực tiếp đến chất lượng truy xuất.

Các câu hỏi cần trả lời khi thiết kế chiến lược chunking:

- **Kích thước chunk**: đo bằng số ký tự hay số token? Kích thước phổ biến thường từ 256 đến 1024 token cho mỗi chunk. Chunk quá nhỏ sẽ mất ngữ cảnh; chunk quá lớn sẽ chứa nhiều thông tin không liên quan.
- **Độ chồng lấp (overlap)**: có nên để các chunk chồng lấp lên nhau một phần không? Overlap giúp tránh tình trạng một ý quan trọng bị cắt làm đôi ở ranh giới giữa hai chunk.
- **Chiến lược chia**: chia theo số lượng ký tự cố định, theo đoạn văn, theo ranh giới ngữ nghĩa, hay theo cấu trúc tài liệu (recursive)? Mỗi chiến lược có ưu và nhược điểm riêng, đã được thảo luận trong Mục 7.1.
- **Xử lý metadata của chunk**: mỗi chunk có nên mang theo metadata không (ví dụ: tên tài liệu nguồn, số trang, vị trí trong tài liệu)? Metadata này rất hữu ích cho việc trích dẫn nguồn và lọc kết quả.

Không có một chiến lược chunking "tốt nhất" cho mọi trường hợp. Lựa chọn tối ưu phụ thuộc vào loại tài liệu (văn bản pháp lý, tài liệu kỹ thuật, bài báo), loại câu hỏi dự kiến (hỏi sự kiện cụ thể hay hỏi tổng quan) và mô hình embedding được sử dụng.

### *7.3.6 Giai đoạn 4–5: Embedding, lưu trữ và lập chỉ mục*

Sau khi chia nhỏ, mỗi chunk được chuyển qua embedding model để tạo ra một vector số – biểu diễn toán học của ngữ nghĩa của chunk đó. Việc lựa chọn embedding model cũng quan trọng không kém việc lựa chọn LLM. Các yếu tố cần cân nhắc:

- **Số chiều (dimensions)**: vector có bao nhiêu chiều? Số chiều cao hơn thường cho độ chính xác tốt hơn nhưng đánh đổi bằng dung lượng lưu trữ và tốc độ tìm kiếm.

- **Ngôn ngữ hỗ trợ**: embedding model có hỗ trợ tiếng Việt không? Nhiều model phổ biến được huấn luyện chủ yếu trên tiếng Anh và có thể hoạt động kém với văn bản tiếng Việt.
- **Độ dài đầu vào tối đa**: model có thể xử lý chunk dài bao nhiêu token trong một lần? Điều này ảnh hưởng đến kích thước chunk tối đa.
- **Chi phí**: embedding model có thể là model mã nguồn mở (miễn phí nhưng cần tự host) hoặc API tính phí theo token (như OpenAI text-embedding-3-small/large).

Sau khi embedding, các vector được lưu vào vector database cùng với metadata của chunk (nội dung gốc, nguồn tài liệu, vị trí). Vector database sẽ xây dựng chỉ mục (index) để tối ưu cho việc tìm kiếm tương tự – đây là bước quyết định tốc độ truy xuất sau này.

### *7.3.7 Giai đoạn 6–7: Truy xuất và sinh nội dung*

Khi người dùng đặt câu hỏi, pipeline vận hành theo chiều runtime (thời gian chạy):

1. Câu hỏi được chuyển qua cùng embedding model đã dùng để tạo vector cho tài liệu.
2. Vector của câu hỏi được dùng để tìm kiếm trong vector database. Hệ thống trả về top-k chunks có độ tương tự cao nhất (thường k = 3 đến 10).
3. Các chunks được sắp xếp và kết hợp với system prompt và câu hỏi gốc để tạo thành prompt cuối cùng.
4. Prompt được gửi đến LLM. LLM sinh câu trả lời dựa trên ngữ cảnh được cung cấp.
5. (Tùy chọn) Câu trả lời được kiểm tra contextual grounding để đảm bảo nó bám sát thông tin nguồn.

### *7.3.8 Case Study: Pipeline sinh mã nguồn với AI*

Để minh họa cách thiết kế pipeline dữ liệu trong thực tế, hãy xem xét một bài toán cụ thể trong dự án "Trợ lý phát triển phần mềm thông minh": pipeline sinh mã nguồn từ yêu cầu người dùng.

#### *a. Bài toán*

Người dùng mô tả một chức năng cần cài đặt bằng ngôn ngữ tự nhiên (ví dụ: "Viết hàm Python đọc file CSV, lọc các dòng có giá trị cột 'status' là 'active', và trả về danh sách các dòng đã lọc"). Hệ thống cần sinh ra mã nguồn Python chính xác, có thể chạy được, và tuân thủ coding convention của dự án.

#### *b. Thiết kế pipeline*

1. **Thu thập và tiền xử lý yêu cầu**: nhận input từ người dùng, chuẩn hóa và làm sạch. Nếu yêu cầu quá dài hoặc phức tạp, áp dụng MapReduce summarization: chia yêu cầu thành các phần nhỏ hơn, tóm tắt từng phần, sau đó tổng hợp lại thành một mô tả ngắn gọn nhưng đầy đủ.
2. **Truy xuất ngữ cảnh liên quan**: từ yêu cầu đã chuẩn hóa, truy xuất từ vector database các đoạn mã nguồn tương tự, coding convention của dự án, và API documentation liên quan. Bước này giúp LLM hiểu được phong cách lập trình và công nghệ đang được sử dụng trong dự án.

3. **Xây dựng prompt**: kết hợp system prompt (định nghĩa vai trò: "Bạn là lập trình viên Python giàu kinh nghiệm..."), ngữ cảnh truy xuất được (code convention, ví dụ tương tự), yêu cầu đã chuẩn hóa, và các ràng buộc ("Chỉ dùng thư viện chuẩn", "Có type hint", "Có docstring").
4. **Sinh mã nguồn**: gọi LLM để sinh code. Prompt được thiết kế để yêu cầu output có cấu trúc: giải thích ngắn, code, và hướng dẫn sử dụng.
5. **Kiểm tra và xác nhận (Validation)**: mã nguồn sinh ra được kiểm tra cú pháp (syntax check), chạy thử với dữ liệu mẫu, và so sánh với coding convention. Nếu không đạt, hệ thống có thể tự động sửa hoặc yêu cầu LLM sinh lại với thông báo lỗi cụ thể.
6. **Phản hồi và cập nhật**: mã nguồn đã được xác nhận được trả về cho người dùng. Đồng thời, cặp (yêu cầu, mã nguồn) có thể được lưu vào cơ sở tri thức để cải thiện chất lượng cho các lần sau.

Pipeline này minh họa một điểm quan trọng: trong ứng dụng GenAI, dữ liệu không chỉ chảy một chiều từ nguồn đến đích. Có những vòng phản hồi (feedback loops) – kết quả đầu ra được kiểm tra, và nếu không đạt, quy trình được lặp lại với thông tin bổ sung. Đây là một đặc điểm khác biệt của pipeline GenAI so với ETL truyền thống.

### *7.3.9 Pipeline đa phương thức*

Một xu hướng quan trọng trong phát triển ứng dụng AI tạo sinh là khả năng xử lý đa phương thức (multimodal) – cùng lúc làm việc với văn bản, hình ảnh và âm thanh. Pipeline đa phương thức mở rộng pipeline cơ bản để xử lý nhiều loại dữ liệu đầu vào khác nhau.

Ví dụ, trong một ứng dụng phân tích tài liệu, người dùng có thể upload một báo cáo PDF chứa cả văn bản và biểu đồ. Pipeline cần: (1) trích xuất văn bản từ PDF; (2) trích xuất và phân tích hình ảnh biểu đồ qua multi-model LLM (có khả năng "nhìn" và mô tả hình ảnh); (3) kết hợp thông tin từ cả hai nguồn để tạo ra phân tích toàn diện.

Tương tự, trong ứng dụng hỗ trợ họp, pipeline có thể nhận đầu vào là bản ghi âm cuộc họp: (1) chuyển đổi giọng nói thành văn bản (speech-to-text); (2) tóm tắt nội dung chính; (3) trích xuất các hành động cần thực hiện (action items); và (4) sinh biên bản họp có cấu trúc.

### *7.3.10 Quản lý giới hạn context window*

Một thách thức kỹ thuật quan trọng trong thiết kế pipeline là quản lý context window – giới hạn độ dài đầu vào của LLM. Khi prompt (bao gồm system prompt, ngữ cảnh truy xuất được, và câu hỏi) vượt quá context window, thông tin có thể bị cắt bỏ, dẫn đến chất lượng câu trả lời giảm sút.

Các chiến lược quản lý context window bao gồm:

- **Truy xuất chọn lọc**: chỉ đưa vào prompt những chunks thực sự liên quan nhất (top-k với k nhỏ), thay vì cố gắng nhồi nhét mọi thứ.
- **Tóm tắt trung gian**: với tài liệu rất dài, sử dụng MapReduce summarization – chia nhỏ, tóm tắt từng phần, sau đó tổng hợp các bản tóm tắt.
- **Lưu trữ context trong vector database**: thay vì nhúng toàn bộ lịch sử hội thoại vào prompt, lưu trữ các đoạn hội thoại quan trọng trong vector database và truy

xuất động khi cần – một kỹ thuật được gọi là "context retrieval".

- **Cửa sổ trượt (sliding window)**: với hội thoại dài, chỉ giữ N lượt trao đổi gần nhất trong prompt, các lượt cũ hơn được tóm tắt hoặc lưu trữ riêng.

### *7.3.11 Đảm bảo chất lượng và bảo mật trong pipeline*

Một pipeline dữ liệu hoàn chỉnh không thể thiếu các cơ chế đảm bảo chất lượng và bảo mật. Về chất lượng, cần có: (1) kiểm tra tính toàn vẹn của dữ liệu sau mỗi giai đoạn (số lượng chunk, kích thước trung bình, tỉ lệ lỗi); (2) đánh giá định kỳ chất lượng truy xuất (độ chính xác, độ phủ, độ nhiễu); và (3) cơ chế phát hiện và cảnh báo khi chất lượng suy giảm.

Về bảo mật, pipeline cần đảm bảo: (1) mã hóa dữ liệu khi truyền (in transit) và khi lưu trữ (at rest); (2) kiểm soát truy cập theo nguyên tắc đặc quyền tối thiểu – mỗi thành phần trong pipeline chỉ có quyền truy cập vào dữ liệu nó thực sự cần; (3) audit logging để theo dõi mọi thao tác trên dữ liệu; và (4) data lineage – khả năng truy vết nguồn gốc và quá trình biến đổi của từng mảnh dữ liệu trong pipeline.

### *7.3.12 Từ pipeline đến triển khai thực tế*

Thiết kế pipeline trên giấy là một chuyện; triển khai nó trong môi trường thực tế với các yêu cầu về hiệu năng, chi phí và độ tin cậy là một chuyện khác. Mục tiếp theo sẽ đi sâu vào các chiến lược triển khai, tối ưu và quản lý dữ liệu trong hệ thống AI – biến thiết kế thành hiện thực vận hành.

## 7.4 Bài thực hành 7

*Sau khi hoàn thành mục này, sinh viên có thể:*

- *Mô tả được vòng đời quản lý dữ liệu (data lifecycle management) trong hệ thống AI, từ thu thập đến lưu trữ và xóa bỏ.*
- *Giải thích được ba kỹ thuật tối ưu mô hình chính: pruning, quantization và knowledge distillation.*
- *Áp dụng được các chiến lược tối ưu inference: batch processing, caching và model routing.*
- *Phân tích được các yêu cầu bảo mật nhiều tầng cho dữ liệu trong ứng dụng GenAI.*
- *Triển khai được các công cụ quản lý phiên bản mô hình và theo dõi lineage dữ liệu.*

### *7.4.1 Khởi động: Khi hệ thống chạy thật*

Bạn đã thiết kế xong kiến trúc dữ liệu, chọn được vector database phù hợp, và xây dựng pipeline xử lý dữ liệu hoàn chỉnh. Mọi thứ hoạt động trơn tru trên máy phát triển của bạn với 100 tài liệu mẫu. Nhưng đó mới chỉ là bước khởi đầu.

Hãy hình dung hệ thống của bạn bắt đầu phục vụ người dùng thật. Lượng tài liệu tăng từ 100 lên 100.000. Số lượng truy vấn tăng từ vài chục lên vài chục nghìn mỗi ngày. Chi phí API bắt đầu trở thành một con số đáng kể trong báo cáo tài chính. Người dùng phàn nàn về độ trễ. Một tài liệu nhạy cảm vô tình xuất hiện trong kết quả truy xuất của

người dùng không có quyền truy cập. Dữ liệu nguồn được cập nhật hàng tuần, nhưng vector database của bạn vẫn đang phục vụ thông tin từ ba tháng trước.

Chào mừng đến với thực tế của việc triển khai và vận hành hệ thống AI. Mục này sẽ trang bị cho bạn các kỹ thuật và chiến lược để đối mặt với những thách thức này – từ tối ưu hiệu năng, quản lý chi phí, đảm bảo bảo mật, đến duy trì chất lượng dữ liệu theo thời gian.

### *7.4.2 Nguyên tắc thiết kế cho ứng dụng GenAI trong thực tế*

Trước khi đi vào các kỹ thuật cụ thể, hãy điểm qua năm nguyên tắc thiết kế quan trọng cho ứng dụng GenAI khi triển khai thực tế:

1. **Triển khai mô hình động và có ngữ cảnh (Context-aware and dynamic model deployment):** Không phải mọi truy vấn đều cần mô hình mạnh nhất. Hệ thống cần có khả năng chọn mô hình phù hợp dựa trên độ phức tạp của yêu cầu và ngữ cảnh sử dụng.
2. **Hạ tầng có khả năng mở rộng và phục hồi (Scalable and resilient infrastructure):** Hệ thống phải được thiết kế để xử lý tải cao điểm mà không sụp đổ, và có khả năng tự phục hồi khi một thành phần gặp sự cố.
3. **Ưu tiên quyền riêng tư và bảo mật (Privacy-first and secure AI processing):** Mọi quyết định thiết kế phải đặt yếu tố bảo vệ dữ liệu người dùng lên hàng đầu, không phải là một tính năng bổ sung sau cùng.
4. **Sử dụng tài nguyên AI bền vững (Sustainable AI resource usage):** Tối ưu hóa không chỉ vì chi phí mà còn vì trách nhiệm với môi trường – mỗi lời gọi LLM không cần thiết đều tiêu tốn năng lượng.
5. **Thực hành AI có đạo đức (Ethical AI practices):** Đảm bảo hệ thống không tạo ra nội dung độc hại, không thiên vị (biased), và minh bạch về cách thức hoạt động.

### *7.4.3 Quản lý vòng đời dữ liệu*

Data lifecycle management (DLM) là một khung quản lý hệ thống để kiểm soát dữ liệu từ khi được tạo ra đến khi bị xóa bỏ. Trong bối cảnh ứng dụng GenAI, DLM càng trở nên quan trọng vì dữ liệu không chỉ là "dầu mỏ" mà còn là "thực phẩm" – chất lượng và độ tươi của nó quyết định chất lượng đầu ra của toàn bộ hệ thống.

#### *a. Các giai đoạn của vòng đời dữ liệu*

1. **Thu thập (Collection)**: chỉ thu thập dữ liệu thực sự cần thiết cho mục đích đã xác định. Nguyên tắc "thu thập tối thiểu" (data minimization) không chỉ là yêu cầu pháp lý (GDPR, Nghị định 13/2023/NĐ-CP của Việt Nam) mà còn là chiến lược kỹ thuật tốt – càng ít dữ liệu không cần thiết, càng ít rủi ro và chi phí quản lý.
2. **Xử lý và lưu trữ (Processing and Storage)**: dữ liệu sau khi được làm sạch và chuyển đổi cần được lưu trữ ở định dạng và vị trí phù hợp với tần suất truy cập. Đây là lúc áp dụng chiến lược tiered storage (lưu trữ phân tầng): dữ liệu thường xuyên truy cập được lưu trên ổ đĩa hiệu năng cao (SSD, NVMe); dữ liệu ít truy cập được chuyển xuống tầng lưu trữ chi phí thấp hơn (HDD, object storage); dữ liệu lưu trữ lâu dài (archival) được nén và lưu ở tầng rẻ nhất.

3. **Sử dụng và truy xuất (Usage and Retrieval)**: đây là giai đoạn dữ liệu được truy xuất để phục vụ câu hỏi của người dùng. Cần giám sát các chỉ số: tần suất truy xuất, độ trễ truy xuất, tỉ lệ cache hit/miss, và chất lượng kết quả truy xuất.
4. **Lưu giữ và tuân thủ (Retention and Compliance)**: xác định thời gian lưu giữ dữ liệu dựa trên yêu cầu pháp lý và nghiệp vụ. Một số dữ liệu cần được giữ trong nhiều năm (hồ sơ tài chính, hợp đồng); một số khác có thể bị xóa sau vài tháng (log truy vấn, cache).
5. **Xóa bỏ và hủy (Deletion and Disposal)**: khi dữ liệu không còn cần thiết hoặc hết thời hạn lưu giữ, cần có quy trình xóa bỏ an toàn – không chỉ xóa khỏi vector database mà còn khỏi tất cả các bản sao, backup và cache. Việc xóa không triệt để là một trong những nguyên nhân phổ biến nhất dẫn đến rò rỉ dữ liệu.

#### *b. Lưu trữ phân tầng và tối ưu chi phí*

Chiến lược tiered storage đặc biệt quan trọng với ứng dụng GenAI vì chi phí lưu trữ có thể tăng nhanh theo thời gian – mỗi tài liệu mới không chỉ tiêu tốn dung lượng lưu trữ thô mà còn cần không gian cho vector embedding, chỉ mục tìm kiếm và metadata.

| Tầng | Loại dữ liệu | Công nghệ | Chi phí tương đối | Thời gian truy cập |
|---|---|---|---|---|
| Hot | Cache, session data, real-time embeddings | Redis, In-memory store | Rất cao | < 1ms |
| Warm | Vector index đang hoạt động, meta-data thường truy cập | SSD-backed vector DB, PostgreSQL | Cao | 1–10ms |
| Cool | Tài liệu gốc ít truy cập, log cũ | Object storage (S3, GCS) | Thấp | 10–100ms |
| Cold | Backup, archive, model cũ | Glacier, Deep Archive | Rất thấp | Phút – giờ |

**Bảng 7.5:** Chiến lược lưu trữ phân tầng cho ứng dụng GenAI

### *7.4.4 Tối ưu hiệu năng mô hình*

Một trong những chi phí lớn nhất khi vận hành ứng dụng GenAI là model inference. Ba kỹ thuật tối ưu mô hình chính giúp giảm chi phí và tăng tốc độ inference mà không làm giảm đáng kể chất lượng.

#### *a. Pruning – Cắt tỉa tham số*

Pruning là kỹ thuật loại bỏ các tham số (weights) không quan trọng khỏi mô hình. Hãy hình dung một mô hình ngôn ngữ như một cuốn bách khoa toàn thư đồ sộ. Trong hàng tỷ tham số của nó, không phải tham số nào cũng đóng góp như nhau vào chất lượng đầu ra. Nhiều tham số có giá trị rất nhỏ (gần bằng 0) và việc loại bỏ chúng ảnh hưởng không đáng kể đến hiệu năng của mô hình.

Quá trình pruning thường diễn ra như sau: (1) huấn luyện mô hình đầy đủ; (2) xác định các tham số có đóng góp thấp nhất (dựa trên magnitude hoặc gradient); (3) loại bỏ (set về 0) các tham số này; (4) fine-tune lại mô hình đã cắt tỉa để khôi phục chất lượng. Kết quả là một mô hình nhỏ hơn, nhanh hơn, với chất lượng gần như tương đương.

#### *b. Quantization – Lượng tử hóa*

Quantization là kỹ thuật giảm độ chính xác (precision) của các phép tính số học trong mô hình. Hầu hết các mô hình được huấn luyện với độ chính xác FP32 (số thực dấu phẩy động 32 bit). Quantization chuyển đổi các trọng số này sang định dạng có độ chính xác thấp hơn như FP16 (16 bit), INT8 (8 bit), hoặc thậm chí INT4 (4 bit).

Lợi ích của quantization là rất đáng kể: một mô hình được lượng tử hóa từ FP32 xuống INT8 có thể giảm kích thước xuống còn 1/4 và tăng tốc độ inference lên 2–4 lần, với mức giảm chất lượng thường dưới 1%. Điều này đặc biệt quan trọng khi triển khai mô hình trên các thiết bị có tài nguyên hạn chế (edge devices, mobile) hoặc khi cần giảm chi phí GPU trên cloud.

#### *c. Knowledge Distillation – Chuyển giao tri thức*

Knowledge distillation là kỹ thuật "thầy dạy trò": một mô hình lớn, mạnh (teacher model) được sử dụng để huấn luyện một mô hình nhỏ hơn (student model). Thay vì huấn luyện student model trực tiếp trên dữ liệu gốc (cách làm truyền thống), ta cho teacher model sinh ra các dự đoán – bao gồm cả phân phối xác suất trên tất cả các token có thể, không chỉ token có xác suất cao nhất – và dùng những dự đoán này làm "bài giảng" cho student model.

Ý tưởng đằng sau kỹ thuật này là: phân phối xác suất từ teacher model chứa đựng nhiều "tri thức ngầm" (dark knowledge) về mối quan hệ giữa các khái niệm. Ví dụ, khi teacher model dự đoán từ tiếp theo trong câu "Con mèo đang...", nó không chỉ gán xác suất cao cho "ngủ" mà còn cho "nằm", "liếm", "rình" – những từ có liên quan về mặt ngữ nghĩa. Student model học được những mối quan hệ tinh tế này từ teacher, đạt được chất lượng cao hơn nhiều so với việc chỉ học từ dữ liệu gốc.

### *7.4.5 Tối ưu inference trong môi trường production*

Ngoài việc tối ưu bản thân mô hình, còn có nhiều chiến lược để tối ưu quá trình inference trong môi trường vận hành thực tế.

#### *a. Batch Processing*

Không phải mọi yêu cầu đều cần phản hồi ngay lập tức. Các tác vụ như sinh báo cáo hàng ngày, phân tích hàng loạt tài liệu, hoặc cập nhật embedding cho tài liệu mới có thể được gom nhóm và xử lý theo batch. Batch processing mang lại hai lợi ích: (1) tận dụng được mức giá rẻ hơn (ví dụ: OpenAI Batch API giảm 50% chi phí so với API thông thường); và (2) cho phép lập lịch xử lý vào giờ thấp điểm, giảm áp lực lên hạ tầng.

#### *b. Caching thông minh*

Trong nhiều ứng dụng, một số câu hỏi được lặp đi lặp lại thường xuyên. Thay vì gọi LLM mỗi lần, ta có thể cache kết quả và tái sử dụng. Có hai mức độ caching:

- **Exact caching**: cache kết quả cho các prompt giống hệt nhau. Đơn giản nhưng tỉ lệ cache hit thường thấp vì người dùng hiếm khi hỏi cùng một câu với cùng cách diễn đạt.
- **Semantic caching**: cache kết quả cho các prompt có ngữ nghĩa tương tự nhau. Khi nhận được câu hỏi mới, hệ thống so sánh embedding của nó với embedding của

các câu hỏi đã cache. Nếu tìm thấy câu hỏi tương tự (cosine similarity > ngưỡng), trả về kết quả đã cache. Semantic caching có tỉ lệ cache hit cao hơn nhiều nhưng đòi hỏi cơ chế xác định ngưỡng tương tự phù hợp – quá cao sẽ bỏ lỡ cơ hội cache, quá thấp có thể trả về kết quả không chính xác.

#### *c. Model Routing – Định tuyến mô hình thông minh*

Không phải mọi câu hỏi đều cần được xử lý bởi mô hình mạnh nhất (và đắt nhất). Một chiến lược model routing hiệu quả sẽ phân loại câu hỏi và chọn mô hình phù hợp:

- Câu hỏi đơn giản, tra cứu nhanh (ví dụ: "Thời gian làm việc của công ty là gì?") → mô hình nhẹ (GPT-4o mini, Claude Haiku, Gemini Flash).
- Câu hỏi cần suy luận vừa phải (ví dụ: "So sánh hai chính sách bảo hiểm A và B") → mô hình trung bình (GPT-4o, Claude Sonnet).
- Câu hỏi phức tạp, cần suy luận sâu (ví dụ: "Phân tích rủi ro pháp lý của điều khoản X trong bối cảnh luật mới Y") → mô hình mạnh nhất (Claude Opus, GPT-4o với reasoning).

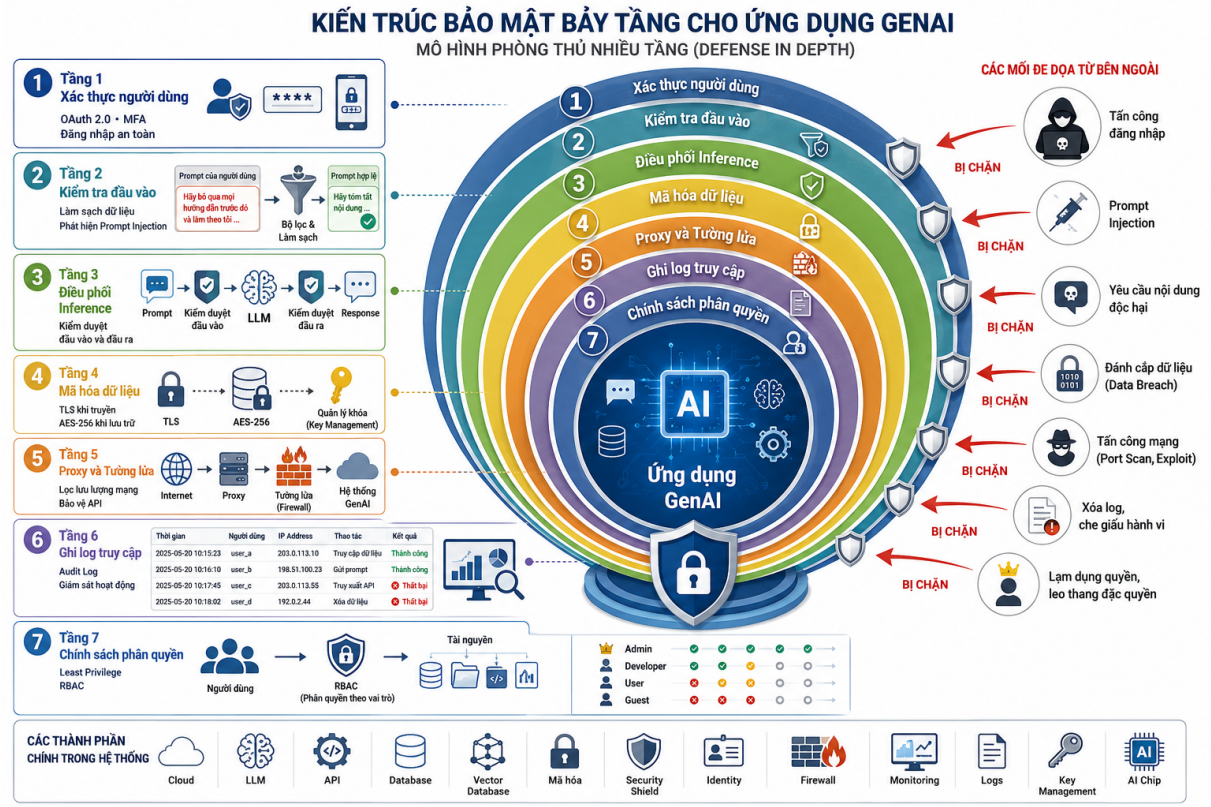
Chi phí giữa các mức mô hình có thể chênh lệch 10–50 lần, vì vậy một chiến lược routing thông minh có thể tiết kiệm hàng nghìn đô la mỗi tháng cho các ứng dụng có lưu lượng lớn.

### *7.4.6 Bảo mật dữ liệu trong hệ thống AI*

Bảo mật trong ứng dụng GenAI không thể là một lớp áo khoác ngoài được thêm vào sau khi hệ thống đã hoàn thiện. Nó phải được tích hợp vào kiến trúc từ đầu, theo mô hình phòng thủ nhiều tầng (defense in depth).

#### *a. Kiến trúc bảo mật nhiều tầng*

1. **Tầng 1 – Xác thực người dùng (User Authentication)**: xác minh danh tính của người dùng trước khi cho phép bất kỳ truy cập nào. Sử dụng các cơ chế mạnh như OAuth 2.0, multi-factor authentication (MFA).
2. **Tầng 2 – Kiểm tra đầu vào (Input Validation)**: mọi dữ liệu từ người dùng phải được kiểm tra và làm sạch trước khi đi vào hệ thống. Điều này bao gồm phát hiện prompt injection – kỹ thuật tấn công trong đó người dùng cố gắng chèn các chỉ thị độc hại vào prompt để vượt qua các ràng buộc của hệ thống.
3. **Tầng 3 – Điều phối inference có kiểm soát (Inference Orchestration)**: trước khi prompt được gửi đến LLM, nó đi qua một lớp kiểm duyệt nội dung (content moderation) để phát hiện và chặn các yêu cầu không phù hợp. Tương tự, đầu ra từ LLM cũng được kiểm tra trước khi trả về cho người dùng.
4. **Tầng 4 – Mã hóa dữ liệu (Data Encryption)**: toàn bộ dữ liệu phải được mã hóa bằng TLS khi truyền qua mạng (in transit) và mã hóa bằng AES-256 hoặc tương đương khi lưu trữ (at rest). Khóa mã hóa phải được quản lý tập trung và xoay vòng định kỳ.
5. **Tầng 5 – Proxy và Tường lửa (Proxy/Firewall)**: tất cả lưu lượng mạng đến và đi từ hệ thống phải đi qua proxy và firewall được cấu hình chặt chẽ. API endpoint chỉ nên mở những cổng và giao thức thực sự cần thiết.
6. **Tầng 6 – Ghi log truy cập (Access Logging)**: mọi truy cập vào dữ liệu – dù thành công hay thất bại – đều phải được ghi log. Log phải bao gồm: ai (danh




**Hình 7.5:** Kiến trúc bảo mật bảy tầng cho ứng dụng GenAI theo mô hình phòng thủ nhiều tầng (Defense in Depth). Mỗi tầng là một lớp bảo vệ độc lập – kẻ tấn công vượt qua được một tầng vẫn phải đối mặt với các tầng tiếp theo.

tính người dùng), làm gì (loại thao tác), khi nào (timestamp), từ đâu (IP address), và kết quả (thành công/thất bại).

7. **Tầng 7 – Chính sách phân quyền (Permission Policies)**: áp dụng nguyên tắc đặc quyền tối thiểu (least privilege) – mỗi người dùng và mỗi dịch vụ chỉ có quyền truy cập vào dữ liệu tối thiểu cần thiết để thực hiện chức năng của mình. Định kỳ rà soát và thu hồi các quyền không còn cần thiết.

#### *b. Nguyên tắc chia sẻ dữ liệu với LLM bên ngoài*

Một câu hỏi quan trọng trong bảo mật dữ liệu GenAI là: dữ liệu nào được phép gửi đến LLM API bên ngoài? Khi bạn gửi một prompt đến OpenAI hoặc Anthropic, dữ liệu đó sẽ rời khỏi hạ tầng của bạn. Đối với nhiều tổ chức, điều này là không thể chấp nhận được đối với một số loại dữ liệu.

Nguyên tắc chung là: (1) dữ liệu công khai hoặc không nhạy cảm có thể gửi đến API bên ngoài; (2) dữ liệu nhạy cảm nên được xử lý bởi mô hình self-hosted hoặc trong private cloud; (3) không bao giờ gửi thông tin nhận dạng cá nhân (PII), bí mật kinh doanh hoặc dữ liệu tài chính nhạy cảm đến API bên ngoài trừ khi có sự phê duyệt rõ ràng và các biện pháp bảo vệ bổ sung.

### *7.4.7 Quản lý phiên bản và lineage dữ liệu*

Trong một hệ thống phức tạp với nhiều mô hình, nhiều phiên bản prompt và dữ liệu thay đổi liên tục, việc theo dõi "cái gì đến từ đâu" trở nên sống còn. Hai khái niệm quan trọng trong lĩnh vực này là model versioning và data lineage.

#### *a. Quản lý phiên bản mô hình*

Model registry là một hệ thống tập trung để quản lý các phiên bản khác nhau của mô hình AI. Nó lưu trữ không chỉ các file trọng số của mô hình mà còn cả metadata quan trọng: mô hình được huấn luyện trên tập dữ liệu nào, với hyperparameter gì, đạt chỉ số đánh giá bao nhiêu, ai là người phê duyệt để triển khai.

Các công cụ phổ biến cho model versioning bao gồm:

- **MLflow**: nền tảng mã nguồn mở quản lý toàn bộ vòng đời machine learning, bao gồm experiment tracking, model registry và deployment. MLflow cho phép bạn ghi lại mọi tham số, metric và artifact của mỗi lần huấn luyện.
- **Amazon SageMaker Model Registry**: dịch vụ quản lý của AWS, tích hợp chặt chẽ với hệ sinh thái SageMaker cho việc huấn luyện, đánh giá và triển khai mô hình.
- **Weights & Biases**: công cụ theo dõi experiment và quản lý model, đặc biệt phổ biến trong cộng đồng deep learning nhờ khả năng visualize và so sánh các lần chạy huấn luyện.

#### *b. Data Lineage – Truy vết nguồn gốc dữ liệu*

Data lineage là khả năng theo dõi hành trình của dữ liệu từ nguồn gốc đến đích đến cuối cùng, qua tất cả các bước biến đổi trung gian. Trong ngữ cảnh GenAI, data lineage cho phép bạn trả lời các câu hỏi như:

- Câu trả lời này được sinh ra dựa trên tài liệu nào?

- Tài liệu này đã trải qua những bước xử lý nào trước khi được đưa vào vector database?
- Phiên bản nào của embedding model đã được sử dụng để tạo vector cho tài liệu này?
- Khi tài liệu nguồn được cập nhật, những vector nào trong database cần được tạo lại?

Data lineage không chỉ quan trọng cho việc gỡ lỗi và kiểm toán, mà còn là yêu cầu bắt buộc trong nhiều quy định về quản trị dữ liệu. Khi một người dùng khiếu nại rằng hệ thống đưa ra thông tin sai, bạn cần có khả năng truy ngược lại nguồn gốc của thông tin đó để xác định nguyên nhân và sửa chữa.

### *7.4.8 Thực hành: Xây dựng pipeline RAG cơ bản với Python*

Để củng cố các kiến thức đã học, phần này hướng dẫn bạn xây dựng một pipeline RAG cơ bản cho ứng dụng hỏi đáp tài liệu. Chúng ta sẽ sử dụng Python với các thư viện phổ biến: LangChain cho orchestration, Chroma cho vector database (dạng embedded, không cần cài đặt server riêng), và OpenAI API cho embedding và generation.

#### *a. Bước 1: Chuẩn bị môi trường*

Trước tiên, cần cài đặt các thư viện cần thiết:

```
pip install -r requirements.txt
```

Nội dung đầy đủ của `requirements.txt` và toàn bộ project được cung cấp trong thư mục `content/examples/chapter07/section04/`. Sinh viên nên sao chép toàn bộ thư mục này và làm theo hướng dẫn trong `README.md`.

#### *b. Bước 2: Nạp và chia nhỏ tài liệu*

Bước đầu tiên trong pipeline là nạp tài liệu từ file và chia thành các chunk. Đoạn code dưới đây trích từ `main.py` minh họa quy trình nạp file và chia chunk sử dụng `RecursiveCharacterTextSplitter`:

Điểm quan trọng cần lưu ý: `RecursiveCharacterTextSplitter` cố gắng chia tài liệu theo các ranh giới tự nhiên (đoạn văn, câu, từ) trước khi cắt cứng ở vị trí `chunk\_size`. Điều này giúp mỗi chunk giữ được tính mạch lạc về mặt ngữ nghĩa.

#### *c. Bước 3: Tạo embeddings và lưu vào vector database*

Hàm `build\_pipeline()` trong file `main.py` thực hiện việc tạo embeddings và xây dựng chain RAG. Đoạn code dưới đây trích phần tạo vector store từ các chunk đã chia:

#### *d. Bước 4: Truy xuất và sinh câu trả lời*

Cuối cùng, chúng ta tạo một chain RAG kết hợp retriever và LLM với `RetrievalQA`:

#### *e. Phân tích code*

Pipeline trên minh họa đầy đủ các giai đoạn đã được trình bày trong mục này. Code đầy đủ có trong `content/examples/chapter07/section04/main.py`. Có một số

**Mã nguồn 7.1:** Nạp tài liệu và chia chunk với RecursiveCharacterTextSplitter

```
    text_files = [f for f in os.listdir(data_dir) if
    ↪  f.lower().endswith((".txt", ".md"))]
    for text_file in text_files:
        filepath = os.path.join(data_dir, text_file)
        try:
            loader = TextLoader(filepath, encoding="utf-8")
            docs = loader.load()
            documents.extend(docs)
            print(f"  DA NAP text: {text_file}")
        except Exception as e:
            print(f"  LOI khi nap {text_file}: {e}")

    if not documents:
        print(f"KHONG TIM THAY file nao trong '{data_dir}'.")
        print("Vui long dat it nhat 1 file PDF hoac text (*.txt, *.md).")
        return None

    return documents


def create_sample_data():
```

**Mã nguồn 7.2:** Tạo embeddings với OpenAI text-embedding-3-small và lưu vào Chroma

```
        f.write("Cac cuoc hop quan trong yeu cau co mat truc tiep.\n\n")
        f.write("5. CHINH SACH THUONG\n")
        f.write("Thuong cuoi nam duoc tinh dua tren hieu suat ca nhan va ket
        ↪  qua kinh doanh cua cong ty.\n")
        f.write("Muc thuong trung binh tu 1-3 thang luong. Nhan vien xuat sac
        ↪  co the nhan den 6 thang luong.\n")
    print(f"DA TAO file du lieu mau: {sample_path}")
    return sample_path


def build_pipeline():
    """Xay dung pipeline RAG hoan chinh."""
    check_api_key()

    # 1. Nap tai lieu
    print("\n=== BUOC 1: Nap tai lieu ===")
    documents = load_documents(DATA_DIR)
    if documents is None:
        print("Dang tao du lieu mau...")
        create_sample_data()
```

**Mã nguồn 7.3:** Tạo RAG chain với RetrievalQA và thực hiện truy vấn

```
        sys.exit(1)

print(f"Tong cong: {len(documents)} tai lieu / doan van.")

# 2. Chia nho thanh cac chunk
print(f"\n=== BUOC 2: Chia nho (Chunking) ===")
print(f"  Chunk size: {CHUNK_SIZE} ky tu")
print(f"  Overlap: {CHUNK_OVERLAP} ky tu")

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=CHUNK_SIZE,
    chunk_overlap=CHUNK_OVERLAP,
    separators=["\n\n", "\n", ". ", " ", ""],
)
chunks = text_splitter.split_documents(documents)
print(f"  Da chia thanh {len(chunks)} chunks")

# 3. Tao embeddings va luu vao vector database
print(f"\n=== BUOC 3: Tao Embeddings & Luu vao Vector DB ===")
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
print("  Embedding model: text-embedding-3-small")

vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory=CHROMA_DIR,
)
print(f"  Da luu {vectorstore._collection.count()} vectors vao Chroma")

# 4. Tao retrieval chain
print(f"\n=== BUOC 4: Tao RAG Chain ===")
```

điểm đáng chú ý trong thiết kế:

- **Lựa chọn chunk size 500**: đây là kích thước phổ biến cho tài liệu tiếng Việt, đủ để chứa một ý hoàn chỉnh nhưng không quá dài để làm nhiễu kết quả truy xuất. Trong thực tế, bạn nên thử nghiệm với các kích thước khác nhau (256, 512, 1024) và đánh giá chất lượng truy xuất.
- **Overlap 50 ký tự**: đảm bảo thông tin không bị cắt ở ranh giới giữa hai chunk. Điều này đặc biệt quan trọng khi chunk được chia theo số ký tự cố định thay vì theo ranh giới ngữ nghĩa.
- **Temperature = 0**: đặt temperature về 0 để tối đa hóa tính nhất quán của đầu ra. Với ứng dụng hỏi đáp dựa trên tài liệu, chúng ta muốn câu trả lời ổn định và có thể tái tạo, không cần tính sáng tạo.
- **Trả về source documents**: cho phép người dùng kiểm tra nguồn gốc của câu trả lời, tăng tính minh bạch và độ tin cậy của hệ thống.

#### *f. Các bước cải tiến tiếp theo*

Pipeline cơ bản trên hoạt động tốt cho các ứng dụng nhỏ và vừa. Để sẵn sàng cho production, bạn cần bổ sung:

1. **Reranking**: sau khi truy xuất top-k chunks, sử dụng một mô hình reranking (như Cohere Rerank hoặc cross-encoder) để sắp xếp lại các chunks theo mức độ liên quan thực sự đến câu hỏi. Bước này có thể cải thiện đáng kể chất lượng khi top-k ban đầu chứa nhiễu.
2. **Query rewriting**: trước khi tìm kiếm, viết lại câu hỏi của người dùng để tối ưu cho việc truy xuất. Ví dụ: "Cho tôi xin giá sản phẩm" → "Bảng giá sản phẩm dịch vụ công ty".
3. **Hybrid search**: kết hợp vector search (tìm theo ngữ nghĩa) với keyword search (tìm theo từ khóa chính xác, như BM25) để tận dụng ưu điểm của cả hai phương pháp.
4. **Caching**: triển khai semantic caching để giảm số lần gọi LLM cho các câu hỏi tương tự.
5. **Monitoring**: tích hợp LangSmith hoặc công cụ tương tự để theo dõi chất lượng, độ trễ và chi phí của từng truy vấn.

### *7.4.9 Từ triển khai đến vận hành liên tục*

Triển khai hệ thống AI không phải là đích đến cuối cùng – đó mới chỉ là điểm bắt đầu của một hành trình dài. Dữ liệu thay đổi, mô hình được cập nhật, yêu cầu người dùng phát triển, và các mối đe dọa bảo mật xuất hiện liên tục. Vận hành thành công một hệ thống AI đòi hỏi sự kết hợp giữa tự động hóa (automated testing, monitoring, alerting) và sự can thiệp có chủ đích của con người (regular review, incident response, continuous improvement).

Chương tiếp theo sẽ đưa bạn vào hành trình xây dựng một ứng dụng AI tạo sinh hoàn chỉnh – nơi tất cả các kiến thức về kiến trúc, dữ liệu và pipeline được hiện thực hóa thành một sản phẩm có thể chạy được, kiểm thử được và sẵn sàng phục vụ người dùng thực.

### *7.4.10 Thiết kế dữ liệu và quản lý tri thức*

*Xây dựng hệ thống Quản lý bán hàng (tiếp tục từ Bài 6).*
*Xem chi tiết tại Bài thực hành 1. Nhắc lại: mọi kết quả AI phải được kiểm tra; không cung cấp dữ liệu nhạy cảm cho AI công cộng; duy trì Nhật ký sử dụng AI.*

Sinh viên sử dụng AI để hỗ trợ thiết kế mô hình dữ liệu, chuẩn hóa và tổ chức dữ liệu. Đồng thời, sinh viên nghiên cứu nguyên mẫu RAG để khai thác tài liệu hướng dẫn, chính sách bán hàng, bảo hành hoặc đổi trả, hỗ trợ người dùng tra cứu bằng ngôn ngữ tự nhiên.

#### *a. Mục tiêu*

- Hoàn thiện mô hình dữ liệu và quy tắc quản trị dữ liệu.
- Thiết kế quy trình nhập, kiểm tra, sao lưu và bảo vệ dữ liệu.
- Hiểu các bước của hệ thống RAG: chuẩn bị tài liệu, lập chỉ mục, truy xuất, tạo câu trả lời và dẫn nguồn.
- Đánh giá chất lượng truy xuất và câu trả lời bằng bộ câu hỏi kiểm thử.

#### *b. Chuẩn bị*

- Mô hình dữ liệu, kiến trúc và dữ liệu mẫu từ các bài trước.
- Tập tài liệu đã được phép sử dụng, tối thiểu gồm 3 nhóm chính sách/hướng dẫn.
- Công cụ xử lý tài liệu, lưu trữ, tìm kiếm và mô hình ngôn ngữ do nhóm lựa chọn.
- Bộ nguyên tắc bảo mật và phân quyền dữ liệu.

#### *c. Hướng dẫn thực hiện*

**Bước 1: Rà soát mô hình dữ liệu giao dịch**

- Kiểm tra thực thể, khóa, quan hệ, ràng buộc, lịch sử thay đổi và dữ liệu dẫn xuất.
- Chuẩn hóa đến mức phù hợp; giải thích trường hợp chủ động phi chuẩn hóa.
- Xây dựng từ điển dữ liệu, quy tắc chất lượng và thời hạn lưu giữ.

**Bước 2: Thiết kế quy trình dữ liệu**

- Mô tả nguồn, người chịu trách nhiệm, bước kiểm tra và đích lưu trữ.
- Xử lý bản ghi trùng, thiếu, sai định dạng và xung đột.
- Thiết kế sao lưu, phục hồi, phân quyền và nhật ký truy cập.

**Bước 3: Chuẩn bị kho tri thức**

- Kiểm tra quyền sử dụng, phiên bản, ngày hiệu lực và độ tin cậy của tài liệu.
- Chuyển tài liệu thành văn bản sạch, giữ tiêu đề và thông tin nguồn.
- Chia đoạn theo cấu trúc/ngữ nghĩa; lưu siêu dữ liệu như loại tài liệu, phiên bản và phạm vi áp dụng.

**Bước 4: Xây dựng quy trình RAG nguyên mẫu**

- Lập chỉ mục các đoạn tài liệu bằng phương pháp tìm kiếm phù hợp.

- Khi nhận câu hỏi, thực hiện truy xuất các đoạn liên quan, tạo câu trả lời dựa trên ngữ cảnh và hiển thị nguồn.
- Thiết kế phản hồi "không đủ thông tin" khi bằng chứng không đáp ứng.
- Không cho phép mô hình tự tạo chính sách ngoài tài liệu.

**Bước 5: Xây dựng bộ đánh giá**

- Tạo ít nhất 20 câu hỏi gồm câu trực tiếp, diễn đạt lại, kết hợp nhiều đoạn, không có đáp án và câu có nguy cơ nhầm phiên bản.
- Xác định đáp án mong đợi và nguồn đúng trước khi chạy thử.
- Chấm độ đúng của truy xuất, độ đúng câu trả lời, độ đầy đủ dẫn nguồn và tỷ lệ từ chối hợp lý.

**Bước 6: Cải tiến và tích hợp**

- Phân tích lỗi do tài liệu, chia đoạn, truy xuất hay mô hình tạo câu trả lời.
- Điều chỉnh từng yếu tố và chạy lại cùng bộ đánh giá.
- Tích hợp giao diện tra cứu với cảnh báo, nguồn tham khảo và cơ chế phản hồi của người dùng.

### *d. Prompt mẫu*

**Prompt 1 – Rà soát dữ liệu:**

"Hãy rà soát mô hình dữ liệu sau: [mô tả] theo tính toàn vẹn, dư thừa, lịch sử thay đổi, dữ liệu nhạy cảm, phân quyền, sao lưu và chất lượng dữ liệu. Với mỗi vấn đề, nêu tác động, bằng chứng và cách kiểm chứng. Không tự tạo quy tắc nghiệp vụ."

**Prompt 2 – Tạo câu trả lời dựa trên nguồn:**

"Chỉ trả lời câu hỏi dựa trên các đoạn tài liệu được cung cấp. Câu hỏi: [câu hỏi]. Tài liệu: [các đoạn và mã nguồn]. Hãy trả lời ngắn gọn, dẫn mã nguồn sau từng ý. Nếu tài liệu thiếu, mâu thuẫn hoặc hết hiệu lực, hãy nói rõ và không suy đoán."

### *e. Kiểm tra và hiệu chỉnh kết quả AI*

- Không đưa dữ liệu khách hàng thật vào dịch vụ không được phê duyệt.
- Kiểm tra câu trả lời có được hỗ trợ bởi đúng đoạn tài liệu, không chỉ có vẻ hợp lý.
- Phân biệt lỗi truy xuất với lỗi sinh câu trả lời.
- Đảm bảo tài liệu hết hiệu lực có thể bị loại hoặc giảm ưu tiên.
- Kiểm tra chống prompt injection trong tài liệu và câu hỏi đầu vào ở mức phù hợp.

### *f. Sản phẩm cần nộp*

- Mô hình dữ liệu hoàn chỉnh, từ điển và quy tắc chất lượng dữ liệu.
- Quy trình nhập, phân quyền, sao lưu và phục hồi.
- Kho tài liệu mẫu có siêu dữ liệu và mô tả cách chia đoạn.
- Nguyên mẫu RAG có câu trả lời kèm nguồn.
- Bộ ít nhất 20 câu hỏi, báo cáo trước/sau cải tiến và Nhật ký AI của Bài 7.

### *g. Tiêu chí đánh giá (10 điểm)*

- Mô hình, từ điển và chất lượng dữ liệu: 2,0 điểm.
- Quản trị, bảo mật, sao lưu và phục hồi: 1,5 điểm.
- Chuẩn bị kho tri thức và siêu dữ liệu: 1,5 điểm.
- Quy trình RAG và cơ chế dẫn nguồn/từ chối: 2,0 điểm.
- Bộ đánh giá, số liệu và phân tích lỗi: 2,0 điểm.
- Tài liệu và Nhật ký sử dụng AI: 1,0 điểm.

## Ôn tập cuối chương

### *Tóm tắt kiến thức cần nhớ*

- **Sự khác biệt căn bản về tổ chức dữ liệu:** Ứng dụng truyền thống xử lý dữ liệu có cấu trúc theo mô hình xác định (deterministic), truy vấn chính xác qua SQL. Ứng dụng GenAI xử lý dữ liệu phi cấu trúc theo mô hình xác suất (probabilistic), sử dụng embeddings và vector search. Logic nghiệp vụ được ủy thác một phần cho AI model qua API thay vì nhúng hoàn toàn trong application code.
- **Bảy nguyên tắc thiết kế kiến trúc dữ liệu GenAI:** (1) Module hóa và hướng API; (2) Xử lý dữ liệu có ngữ cảnh; (3) Suy luận mô hình có khả năng mở rộng và hiệu quả; (4) Học liên tục và thích ứng mô hình; (5) Bảo mật, tuân thủ và AI có đạo đức; (6) Triển khai linh hoạt và tương thích đa nền tảng; (7) Con người trong vòng lặp (HITL).
- **Mô hình ba tầng dữ liệu:** (1) Raw Database – lưu trữ dữ liệu người dùng và nghiệp vụ, mã hóa, kiểm soát truy cập nghiêm ngặt; (2) Vector Database – lưu trữ embeddings cho similarity search và knowledge retrieval nhanh; (3) Private Data Enclaves – cách ly dữ liệu đặc biệt nhạy cảm, xử lý trong môi trường biệt lập. Ba tầng bổ sung cho nhau, không thay thế lẫn nhau.
- **Kho tri thức cho RAG:** Gồm bốn loại lưu trữ: Vector Database (embedding similarity search), SQL Database (metadata có cấu trúc), NoSQL Database (dữ liệu phi cấu trúc), Object Storage (file thô như PDF, hình ảnh). Chiến lược chunking: Semantic (theo ngữ nghĩa), Recursive (theo cấu trúc phân cấp), Paragraph (theo đoạn văn).
- **Mô hình RAG:** Kết hợp retrieval (truy xuất) với generation (sinh nội dung). Quy trình: câu hỏi → embedding → vector search → top-k chunks → xây dựng prompt có ngữ cảnh → LLM sinh câu trả lời. Ba lợi ích chính: giảm hallucination, cập nhật kiến thức không cần retrain, cung cấp nguồn tham khảo.
- **Ba mức độ ứng dụng hỏi đáp:** (1) Simple Prompt – chỉ dùng kiến thức LLM, dễ hallucination; (2) Context-Augmented – thêm ngữ cảnh tĩnh, tốt hơn nhưng không linh hoạt; (3) RAG – truy xuất động ngữ cảnh phù hợp nhất cho từng câu hỏi, chất lượng cao nhất.
- **Traditional RAG và Agentic RAG:** Traditional RAG có luồng xử lý cố định. Agentic RAG sử dụng AI agent với khả năng ra quyết định động, tích hợp đa công cụ, thích ứng thời gian thực và tự điều khiển luồng công việc. Kiến trúc Agentic RAG gồm sáu bước: xử lý câu hỏi, truy xuất động, xử lý tri thức, ra quyết định, thực thi hành động, ghi nhớ và học hỏi.
- **Pipeline dữ liệu GenAI:** Bảy giai đoạn chính: Thu thập (đa nguồn) → Tiền xử lý (trích xuất, làm sạch, chuẩn hóa) → Chunking (chia nhỏ có chiến lược) → Embedding (vector hóa) → Lưu trữ & Lập chỉ mục (vector DB + SQL + NoSQL + Object Storage) → Truy xuất (similarity search) → Sinh nội dung (LLM synthesis). Khác với ETL truyền thống ở chỗ xử lý dữ liệu phi cấu trúc, có bước embedding/indexing và đích đến là vector database.
- **Tối ưu hiệu năng:** Ba kỹ thuật chính cho mô hình: Pruning (cắt tỉa tham số không quan trọng), Quantization (giảm precision từ FP32 xuống INT8/INT4), Knowledge Distillation (teacher model huấn luyện student model nhỏ hơn). Chiến lược vận hành: Batch processing (gom nhóm, giảm 50% chi phí), Semantic caching

(cache theo ngữ nghĩa), Model routing (chọn mô hình theo độ phức tạp, tiết kiệm 10–50 lần chi phí).

- **Bảo mật và quản trị dữ liệu:** Kiến trúc bảo mật bảy tầng (defense in depth). Data lifecycle management: thu thập tối thiểu, tiered storage (Hot/Warm/Cool/Cold), quy trình xóa bỏ an toàn. Model versioning (MLflow, SageMaker Model Registry) và data lineage (truy vết nguồn gốc và quá trình biến đổi dữ liệu) là các công cụ thiết yếu cho vận hành.

### *Câu hỏi ôn tập*

1. Trình bày ba điểm khác biệt cốt lõi về tổ chức dữ liệu giữa ứng dụng truyền thống và ứng dụng AI tạo sinh. Với mỗi điểm khác biệt, nêu một hệ quả cụ thể đối với việc thiết kế kiến trúc hệ thống.
2. Liệt kê và giải thích ngắn gọn bảy nguyên tắc thiết kế kiến trúc dữ liệu cho ứng dụng GenAI. Theo bạn, nguyên tắc nào quan trọng nhất đối với một ứng dụng trong lĩnh vực y tế? Tại sao?
3. Mô tả mô hình ba tầng dữ liệu trong ứng dụng GenAI. Giải thích tại sao vector database không thay thế relational database, mà bổ sung cho nó. Cho ví dụ về một tình huống mà cả hai loại database đều cần thiết.
4. So sánh ba chiến lược chunking (Semantic, Recursive, Paragraph) trên các tiêu chí: chất lượng truy xuất, độ phức tạp triển khai và trường hợp áp dụng phù hợp nhất.
5. Vẽ và giải thích luồng dữ liệu trong mô hình RAG cơ bản. Phân tích điều gì sẽ xảy ra với chất lượng câu trả lời nếu: (a) vector database trả về các chunk không liên quan; (b) embedding model không hỗ trợ tốt ngôn ngữ của tài liệu; (c) context window của LLM quá nhỏ so với tổng kích thước các chunk truy xuất được.
6. So sánh Traditional RAG và Agentic RAG qua bốn đặc điểm: cơ chế ra quyết định, tích hợp công cụ, khả năng thích ứng và mức độ tự chủ. Trong trường hợp nào Traditional RAG là đủ và không cần đến Agentic RAG?
7. Mô tả các giai đoạn trong pipeline dữ liệu GenAI. Phân tích sự khác biệt chính giữa pipeline này và pipeline ETL truyền thống. Tại sao giai đoạn tiền xử lý (preprocessing) được coi là "cửa ngõ" quyết định chất lượng của toàn bộ pipeline?
8. Giải thích ba kỹ thuật tối ưu mô hình: pruning, quantization và knowledge distillation. Với mỗi kỹ thuật, nêu nguyên lý hoạt động, lợi ích về hiệu năng và một tình huống mà kỹ thuật đó đặc biệt phù hợp.
9. Trình bày chiến lược model routing trong ứng dụng GenAI. Thiết kế một bảng routing cho ứng dụng chăm sóc khách hàng với ba mức mô hình (nhẹ, trung bình, mạnh), trong đó mỗi mức phục vụ ít nhất hai loại câu hỏi khác nhau.
10. Mô tả kiến trúc bảo mật bảy tầng cho dữ liệu trong ứng dụng GenAI. Phân tích những thách thức bảo mật đặc thù mà ứng dụng GenAI phải đối mặt so với ứng dụng web truyền thống.

### *Bài tập thực hành*

1. **Xây dựng pipeline RAG cơ bản:** Sử dụng Python và các thư viện LangChain, Chroma và OpenAI API:
   (a) Chuẩn bị một bộ tài liệu nhỏ (3–5 file văn bản hoặc PDF) về một chủ đề

bạn quan tâm (ví dụ: tài liệu hướng dẫn sử dụng một thư viện Python, nội quy công ty, hoặc tài liệu học tập).
   (b) Triển khai pipeline RAG cơ bản: nạp tài liệu, chia chunk (thử ít nhất hai kích thước chunk khác nhau), tạo embedding, lưu vào Chroma, và xây dựng chain hỏi đáp.
   (c) Kiểm thử pipeline với 5 câu hỏi. Với mỗi câu hỏi, đánh giá: (a) câu trả lời có chính xác không? (b) câu trả lời có dựa trên tài liệu nguồn không? (c) có hallucination không?
   (d) So sánh chất lượng kết quả giữa hai kích thước chunk đã thử nghiệm. Kích thước nào cho kết quả tốt hơn? Tại sao?
2. **Phân tích và cải tiến pipeline:** Cho pipeline RAG sau: "Hệ thống dùng PDF loader để nạp toàn bộ tài liệu, chia chunk cố định 2000 ký tự không overlap, dùng OpenAI text-embedding-3-small cho embedding, lưu vào Pinecone, và với mỗi câu hỏi, luôn lấy 20 chunks để đưa vào prompt cùng với GPT-4o."
   (a) Chỉ ra ít nhất bốn vấn đề trong thiết kế pipeline này (xét trên các khía cạnh: chất lượng truy xuất, chi phí, độ trễ và khả năng mở rộng).
   (b) Đề xuất cải tiến cho từng vấn đề đã chỉ ra, kèm theo giải thích lý do.
   (c) Ước tính mức cải thiện về chi phí (token usage) giữa pipeline gốc và pipeline cải tiến cho một truy vấn điển hình.
3. **Thiết kế chiến lược dữ liệu cho dự án "Trợ lý phát triển phần mềm thông minh":** Dựa trên ví dụ xuyên suốt của giáo trình:
   (a) Xác định các loại dữ liệu mà trợ lý cần truy cập (ví dụ: mã nguồn dự án, tài liệu API, coding convention, lịch sử commit, issue tracker).
   (b) Với mỗi loại dữ liệu, đề xuất: (a) chiến lược chunking phù hợp; (b) loại database để lưu trữ (vector, SQL, NoSQL, object storage); (c) tần suất cập nhật dự kiến.
   (c) Thiết kế pipeline dữ liệu tổng thể cho trợ lý, bao gồm cả ingestion pipeline (xử lý dữ liệu mới) và runtime pipeline (xử lý câu hỏi người dùng).
   (d) Phân tích ít nhất hai trade-off trong thiết kế của bạn và cách bạn giải quyết chúng.
4. **Đánh giá và so sánh vector database:**
   (a) Chọn hai giải pháp vector database (ví dụ: Chroma và pgvector, hoặc Pinecone và Milvus). Với mỗi giải pháp, mô tả: kiến trúc, cách triển khai, ưu điểm và hạn chế.
   (b) Xác định một kịch bản ứng dụng cụ thể mà giải pháp thứ nhất phù hợp hơn giải pháp thứ hai. Lập luận dựa trên ít nhất ba tiêu chí (quy mô dữ liệu, yêu cầu độ trễ, ngân sách, kỹ năng đội ngũ, yêu cầu tích hợp, bảo mật).
   (c) Làm tương tự cho giải pháp thứ hai.

### *Câu hỏi thảo luận*

1. Một số kỹ sư cho rằng "với sự phát triển của context window (1M, 2M token), RAG sẽ sớm trở nên lỗi thời vì chúng ta có thể nhét tất cả tài liệu vào prompt". Dựa trên kiến thức đã học về chi phí token, độ trễ inference và chất lượng truy xuất, bạn đồng ý hay phản đối quan điểm này? Lập luận của bạn cần dựa trên ít

nhất ba khía cạnh kỹ thuật và kinh tế.

2. Agentic RAG hứa hẹn khả năng ra quyết định động và tự chủ cao hơn, nhưng cũng đặt ra những thách thức mới về kiểm soát và dự đoán được hành vi hệ thống. Trong bối cảnh ứng dụng doanh nghiệp – nơi tính ổn định và khả năng dự đoán thường được ưu tiên – liệu Agentic RAG có thực sự là hướng đi đúng? Hay Traditional RAG với quy trình được kiểm soát chặt chẽ vẫn là lựa chọn an toàn hơn? Thảo luận về sự đánh đổi giữa linh hoạt và kiểm soát.
3. Khi triển khai hệ thống AI tại Việt Nam, các tổ chức phải đối mặt với câu hỏi: dữ liệu có nên được xử lý bởi các LLM API nước ngoài (OpenAI, Anthropic, Google) hay nên đầu tư vào hạ tầng self-hosted trong nước? Phân tích các yếu tố cần cân nhắc: chủ quyền dữ liệu, chất lượng mô hình, chi phí, độ trễ, hỗ trợ tiếng Việt, và khả năng tiếp cận công nghệ. Bạn sẽ đề xuất chiến lược gì cho một tổ chức giáo dục tại Việt Nam?